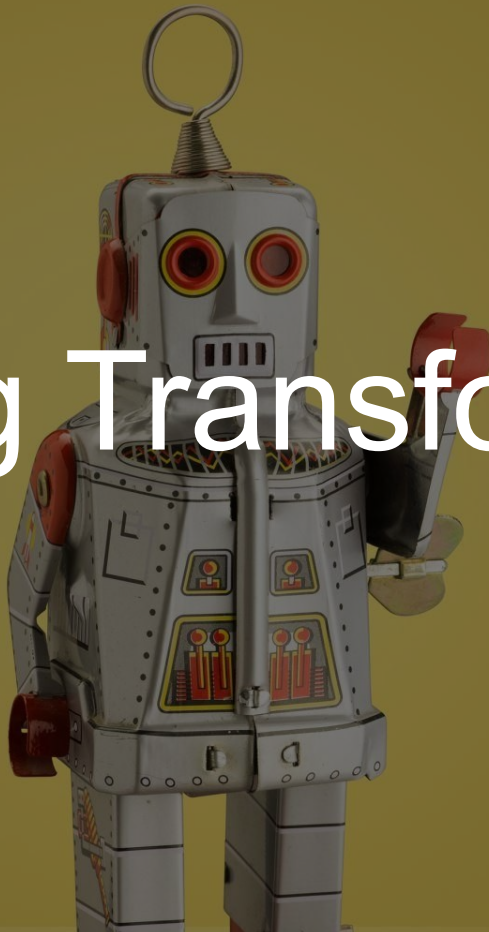


Join the lecture online on your dashboard.

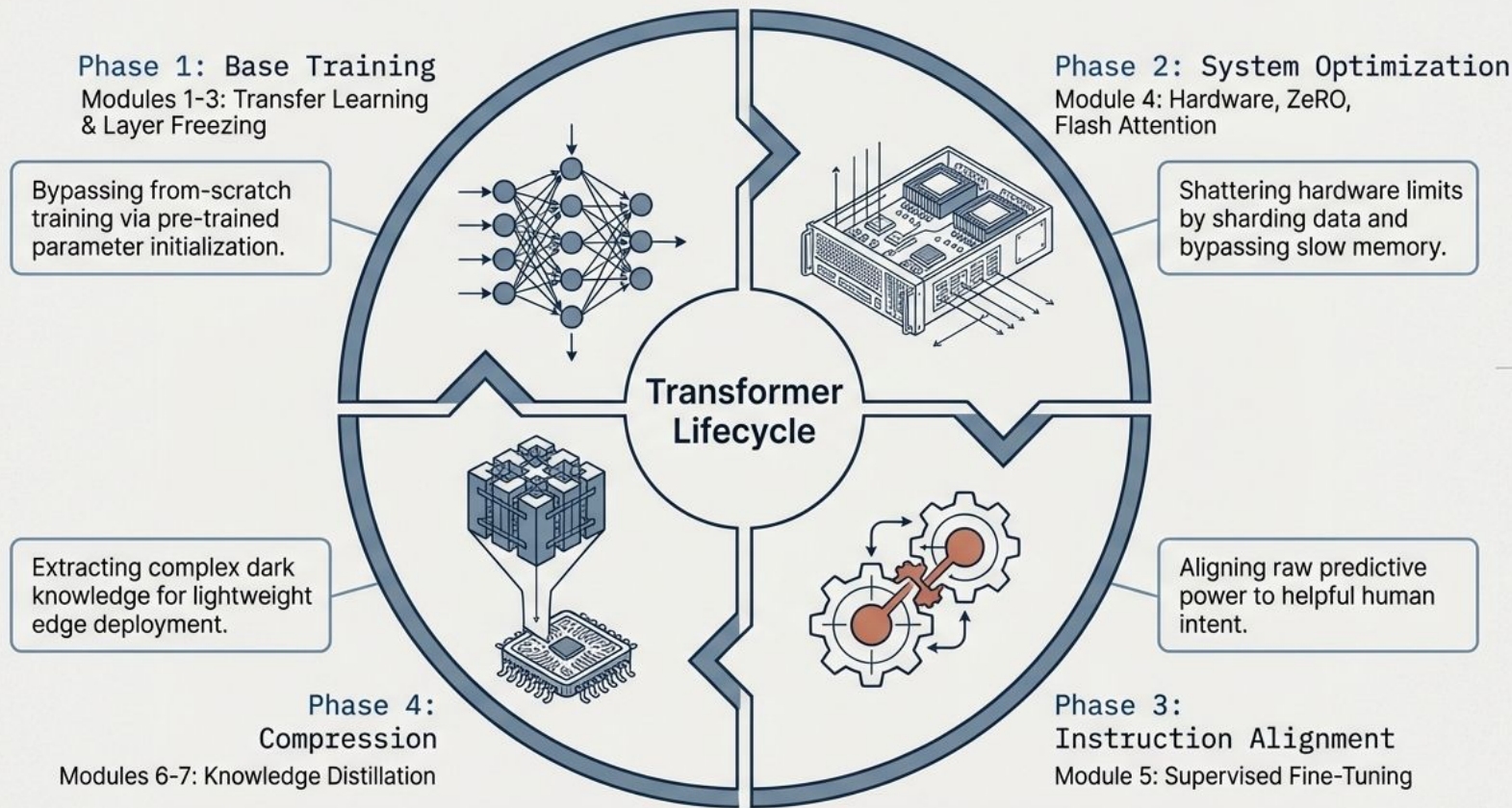
Training Transformers



LECTURE TOPICS: TRANSFORMER ARCHITECTURE, APPLICATIONS & MODIFICATIONS

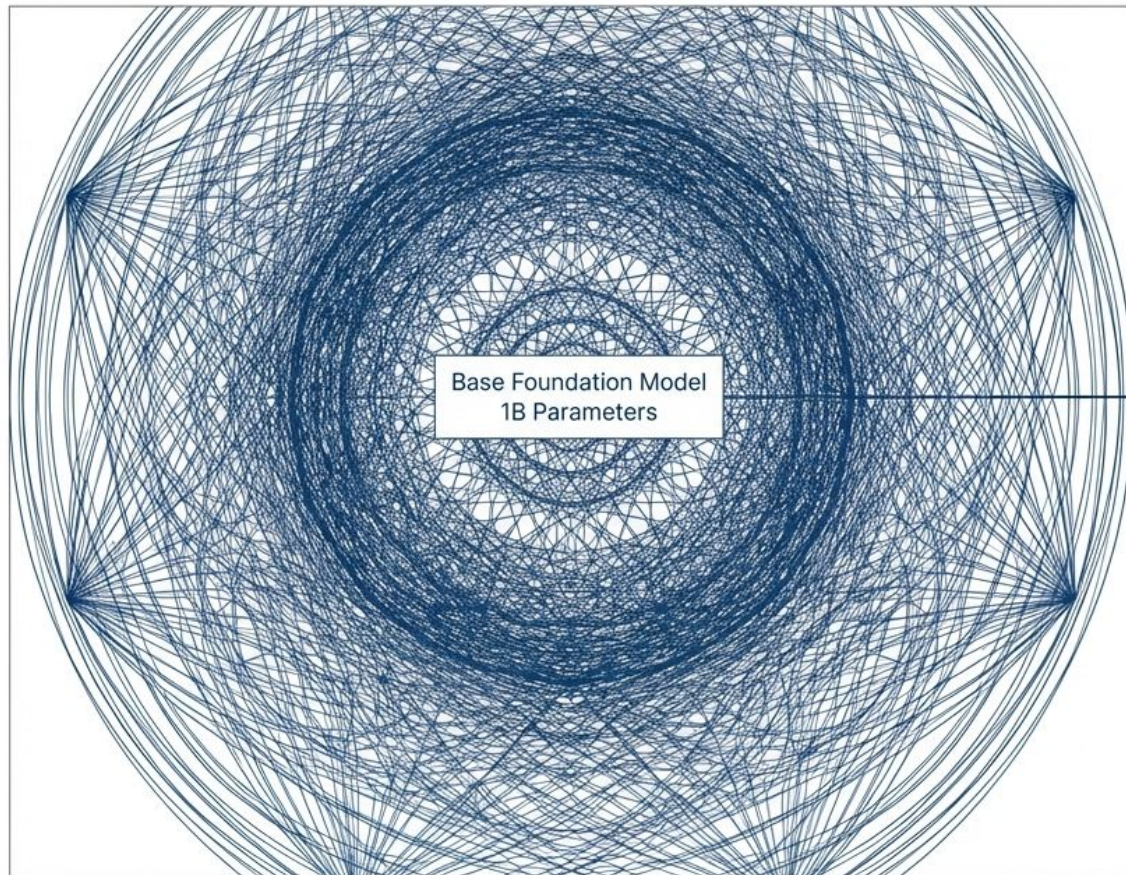
- I. Introduction to Transfer Learning (Recap, Concepts, Fine-tuning, Multi-task)
- II. Transformer Training Lifecycle (Pre-training, Fine-tuning, Inference, Scaling Laws)
- III. Training Setup and Infrastructure (LLM Setup, Optimizer, Parallelism Types)
- IV. Performance Optimization (Flash Attention vs. Standard Attention, Hardware-aware Optimization)
- V. Advanced Optimization and Infrastructure (LLM-Specific Sharding, Distributing Models Across GPUs, Flash Attention Implementation)
- VI. Specialized Adaptations and Efficiency (Hyperparameter Tuning for Transformers, Adaptive Compute, Pruning)
- VII. Fine-tuning and Adaptation (Supervised Fine-tuning, Instruction Tuning, Evaluation Benchmarks, PEFT/LoRA)

The AI Optimization Funnel: From Raw Scale to Refined Utility



Recap : I. Transfer Learning

The Baseline Computational Constraint

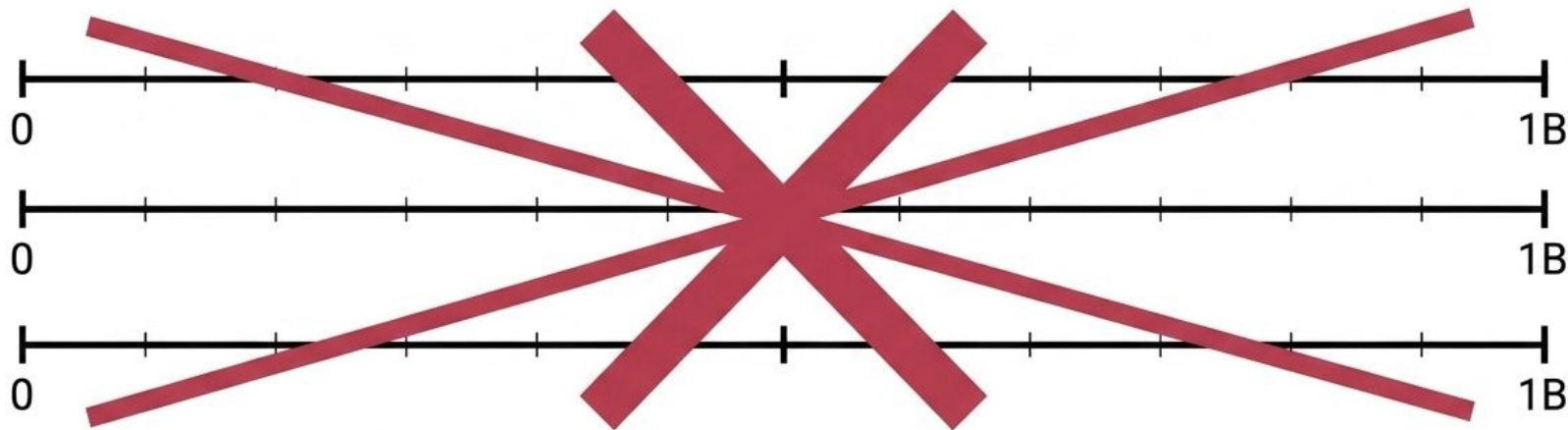


The foundational requirement begins with a monolithic architecture: a **1 Billion parameter** transformer.

Question I: You have deployed a 1-Billion parameter transformer model. The market now demands three highly specialized iterations: a Legal assistant, a Medical assistant, and a Code assistant. Do you build three new models from scratch?

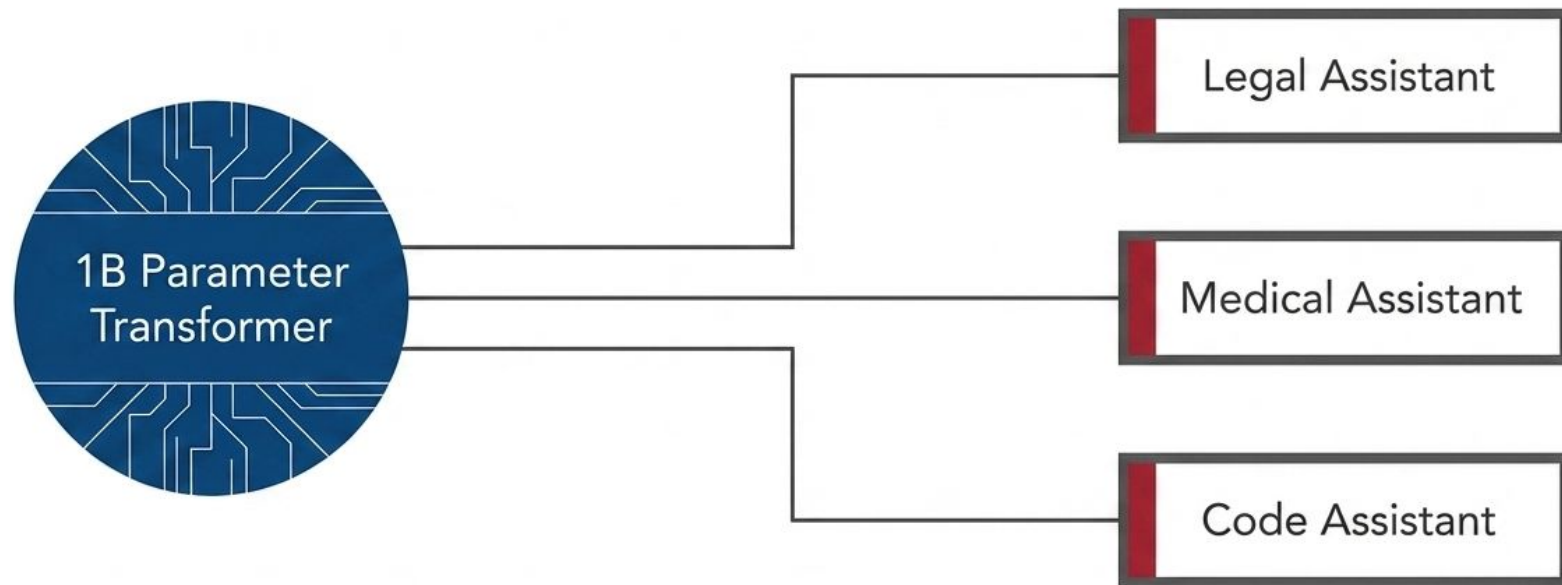


Answer I: No. The computational overhead of training from scratch is economically and operationally prohibitive.



**[1B Parameter Model] × [3 Domains]
= [3x Redundant Compute Costs]**

The Divergence of Specialized Assistants

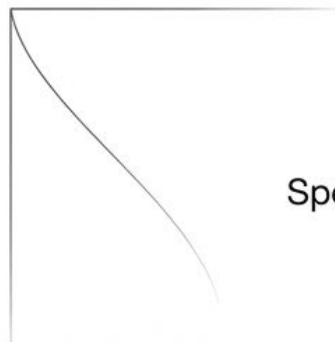


Deployment requires fracturing the baseline model into distinct, highly specialized operational agents.

Question II: If foundational training from scratch is obsolete, how do we bridge the gap between generalized intelligence and specialized domain expertise?



Generalization



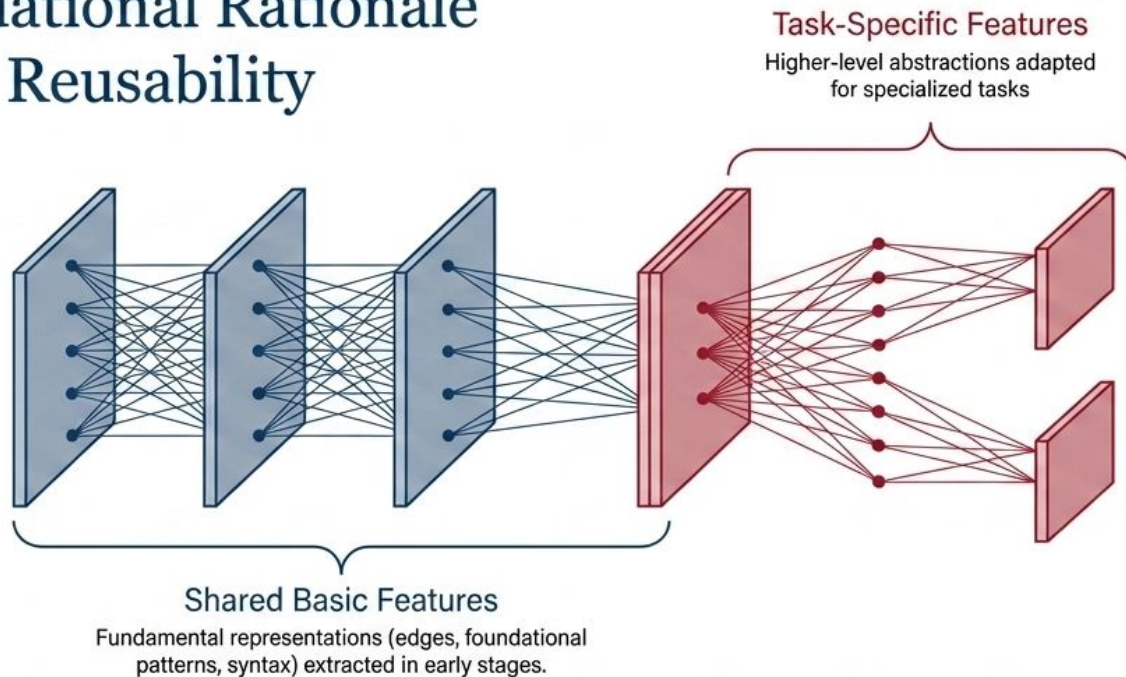
Specialization

Answer II: The Transfer Learning Paradigm. We extract and migrate intelligence from a model trained on Source Data A to Target Data B.



Question III: Why would a model trained on generic, generalized source data hold any predictive value or utility for highly specialized target environments?
environments?

The Foundational Rationale of Feature Reusability



Similar algorithmic features remain highly useful even across distinct, differentiated tasks.

Fundamental representations (edges, foundational patterns, syntax) are shared by different, yet inherently related, data structures.

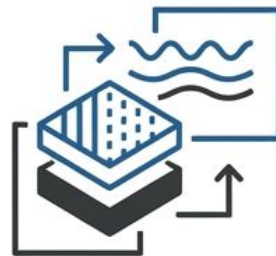
Question IV: Is this transfer process a monolithic, one-size-fits-all approach, or does it dynamically adapt based on the specific constraints of the target environment?

Answer IV: The architecture bifurcates into two distinct operational possibilities.

Task Transfer



Data Transfer

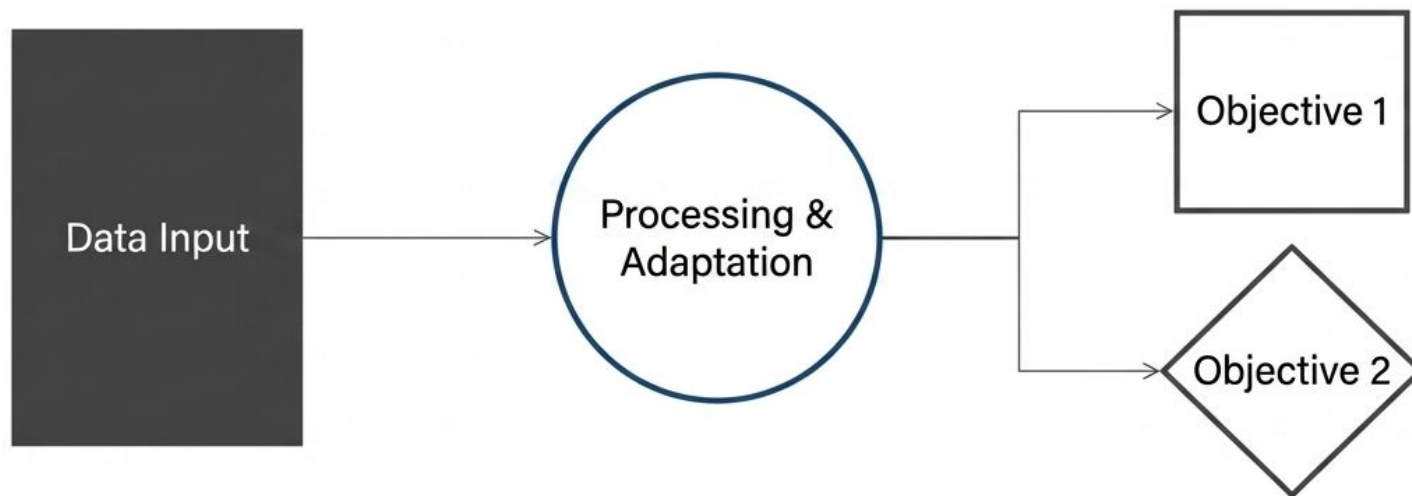


Question V: What is the optimal strategy when we possess the exact same underlying data source, but our analytical objective shifts entirely?

Answer V: Task Transfer. The source and target data can be the same; the mathematical objective evolves.

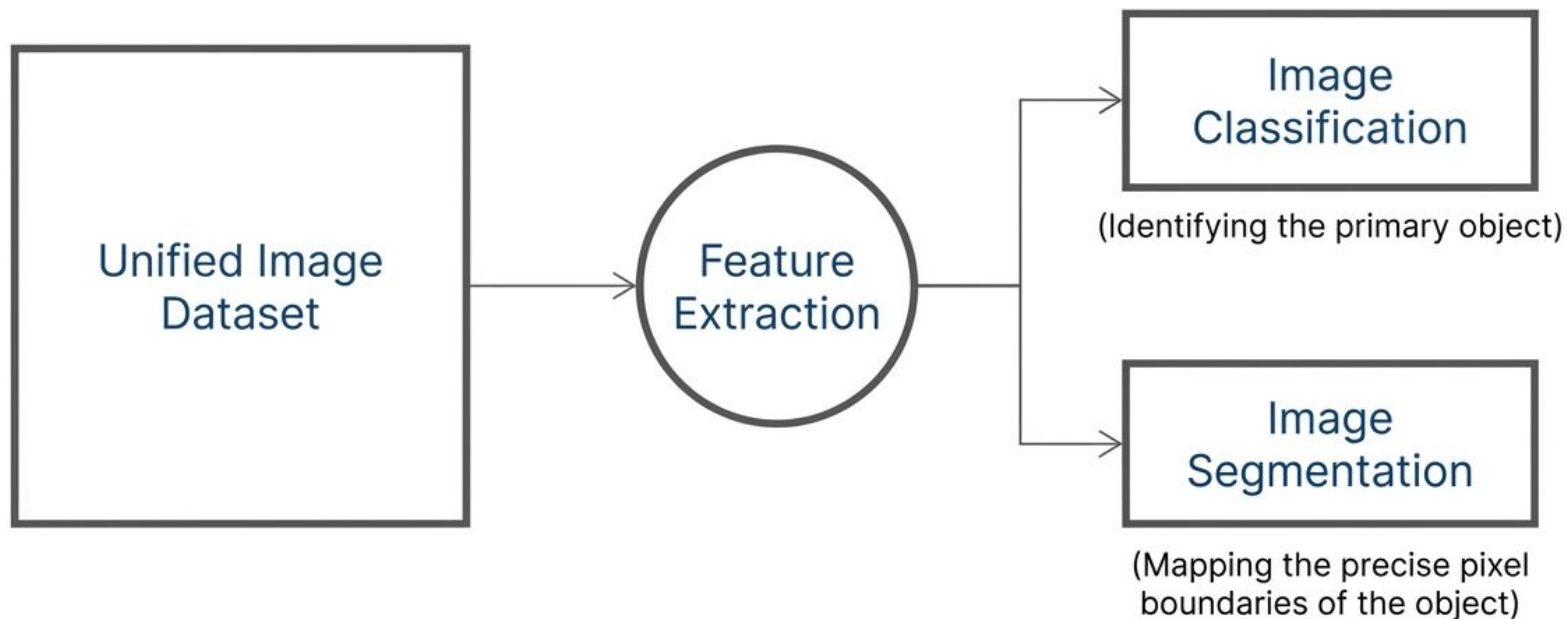
Input Data	Transformation	Output Objective
	Image classification → Image segmentation	
	Machine translation → Sentiment analysis	
	Time series prediction → Time series classification	

Task Transfer: Preserving Data While Shifting Objectives



- Constant Variable: The Source Data and Target Data remain identical.
- Dynamic Variable: The operational mechanism shifts to a novel objective utilizing the same underlying corpus.

Architectural Routing in Visual Task Transfer



Applied Frameworks for Algorithmic Task Shift

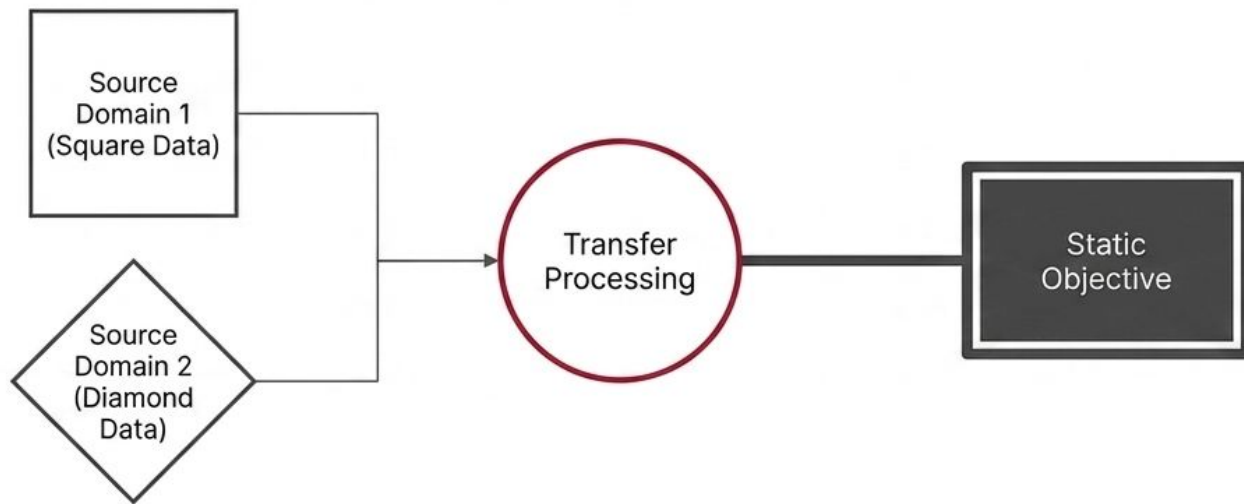
UNIFIED SOURCE DATA	SOURCE TASK	TARGET TASK
Multilingual Text Corpus	Machine Translation (Mapping syntax across languages)	Sentiment Analysis (Extracting emotional valence from the same text)
Sequential Temporal Data	Time Series Prediction (Forecasting future data points)	Time Series Classification (Categorizing the temporal pattern)

Question VI: Conversely, what is the strategy when our operational objective remains strictly constant, but the underlying data environment changes completely?

Answer VI: Data Transfer. **The objective** remains constant; **the environmental input shifts**.

Input Data	Transformation	Output Objective
	Images of every objects → Medical images	
	Chinese text → English text	
	Physiological signals of one patient → Another patient	

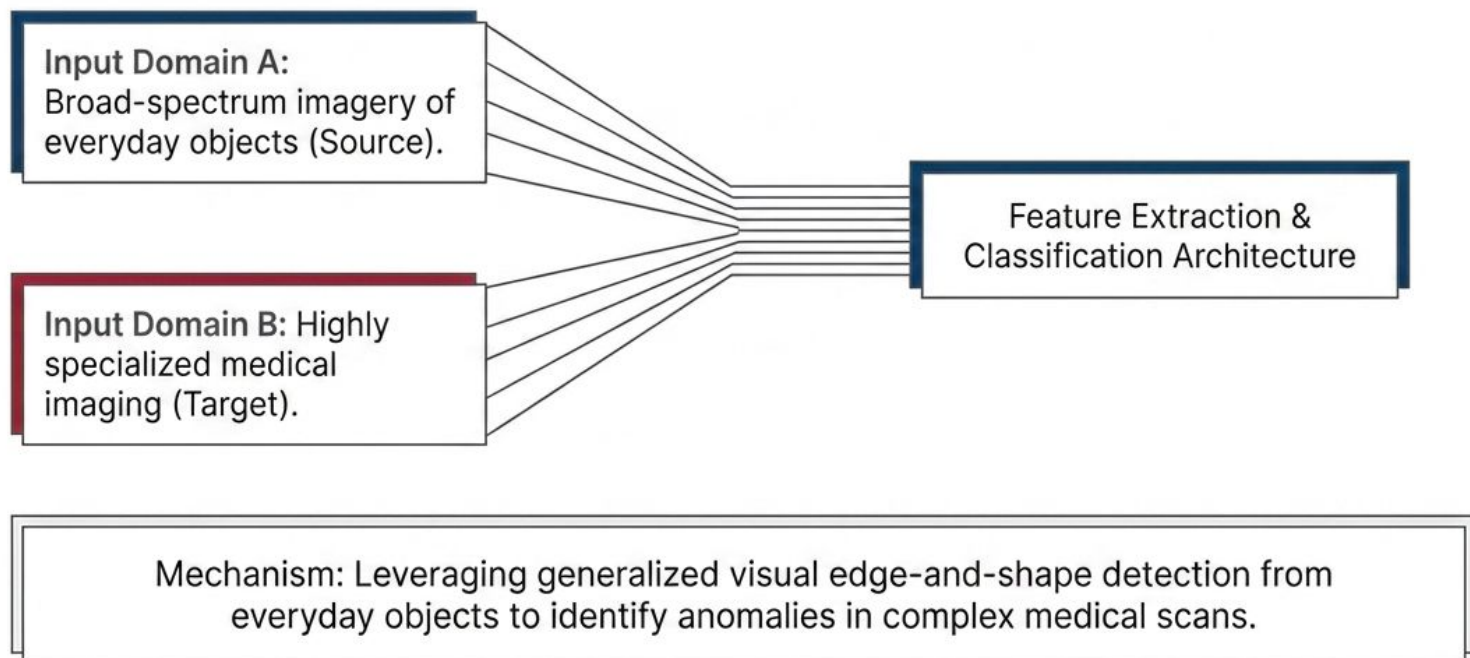
Data Transfer: Preserving Objectives Across Novel Domains



■ Constant Variable: The analytical task and output requirement remain identical.

□ Dynamic Variable: The underlying source data domain shifts significantly to a novel target data domain.

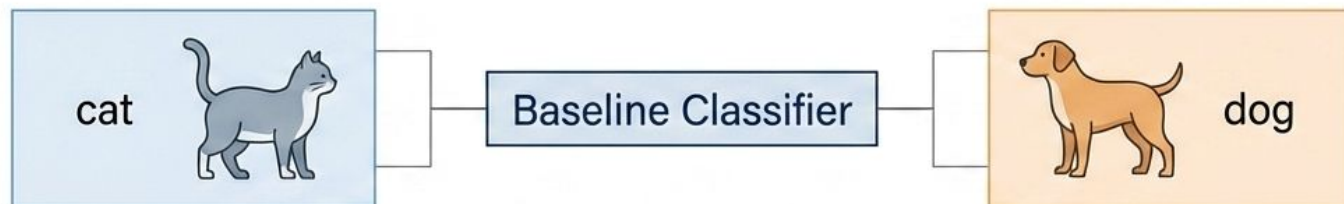
Architectural Routing in Visual Domain Shift



Applied Frameworks for Underlying Domain Shift

Unified Objective	Source Domain	Target Domain
Linguistic Parsing and Generation	Chinese language corpus	English language corpus
Biological Anomaly Detection	Physiological signals of Patient A	Physiological signals of Patient B

Navigating Indirect Data Relationships



Baseline Condition: A foundational classifier trained explicitly on domestic biological subjects (Dog/Cat).

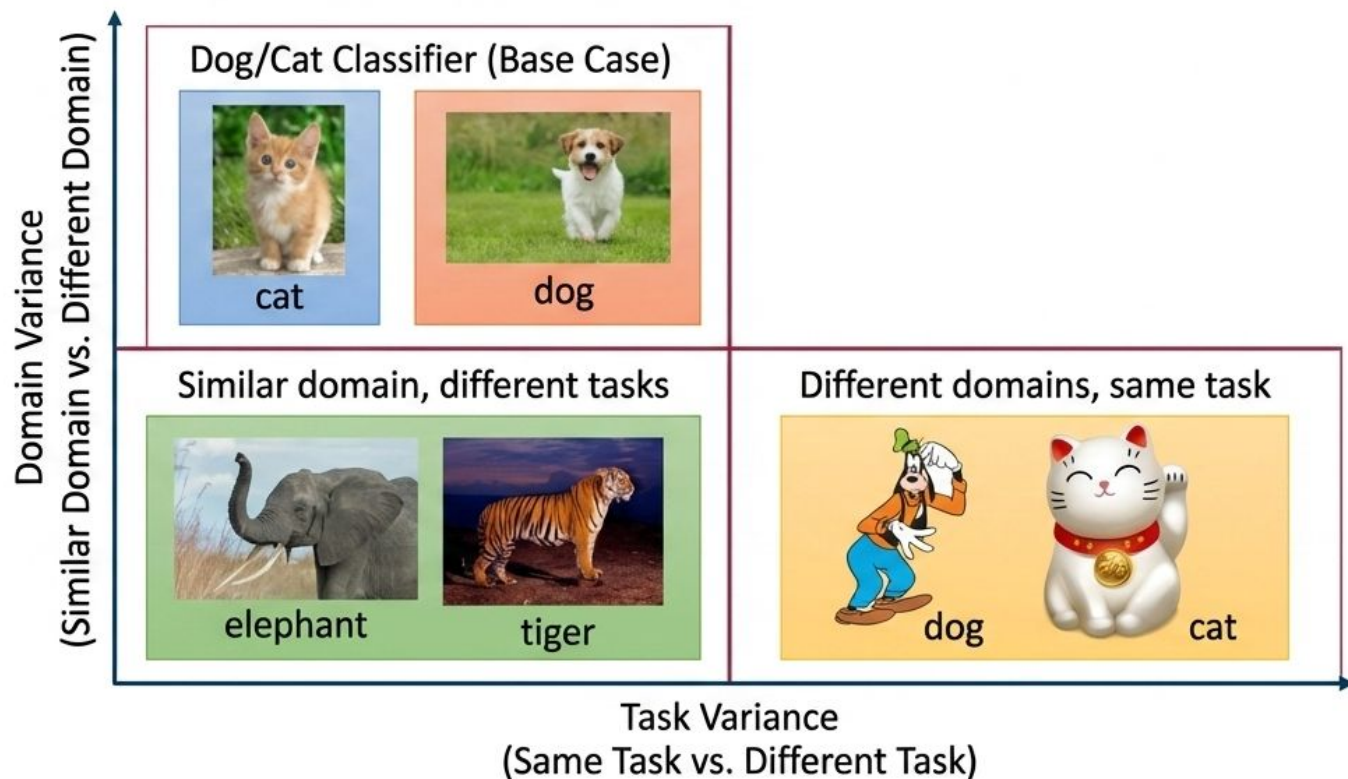
Data Not Directly Related

The Analytical Challenge:

Evaluating transfer viability when presented with data that is not directly related to the baseline task.

Question VII: How do we visualize and operationalize these distinct transfer pathways into a unified, actionable strategic framework?

Answer VII: The Domain-Task Matrix. A diagnostic tool for mapping transfer viability across related datasets.



The Domain-Task Transfer Matrix

		Domain	
		Similar Domain	Different Domain
Task	Same Task		 The domain shifts radically (illustrations/sculpture), but the fundamental task (identifying dog vs. cat features) remains constant.
	Different Task	 The domain (real biological animal photography) matches the dog/cat baseline, but the target categorization task is entirely new.	

II. Training Lifecycle

MODULE II

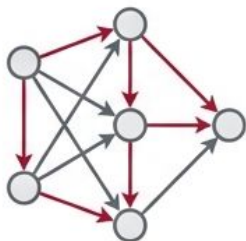
The Training Lifecycle & Scaling Laws

From Transfer Learning to Large
Language Model Economics

The Evolution of Model Training

Phase 1: Knowledge Transfer

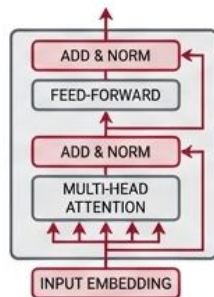
Optimizing Parameters



- Transfer Learning
- Multi-Task Learning

Phase 2: The Transformer Shift

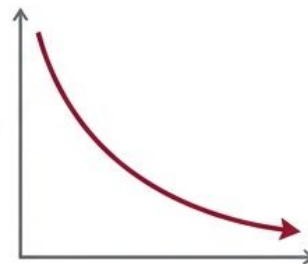
Learning Generalized Patterns



- Pre-training
- Fine-tuning

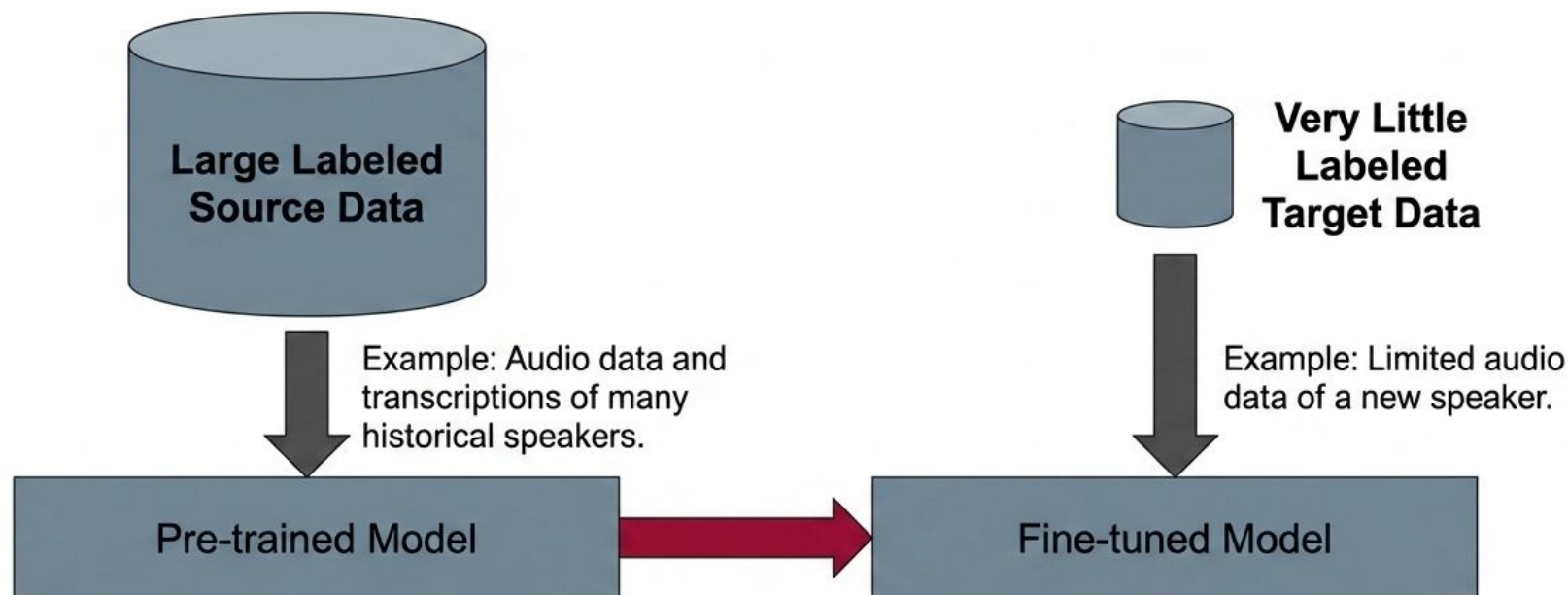
Phase 3: The Era of Scale

Optimizing Compute and Data

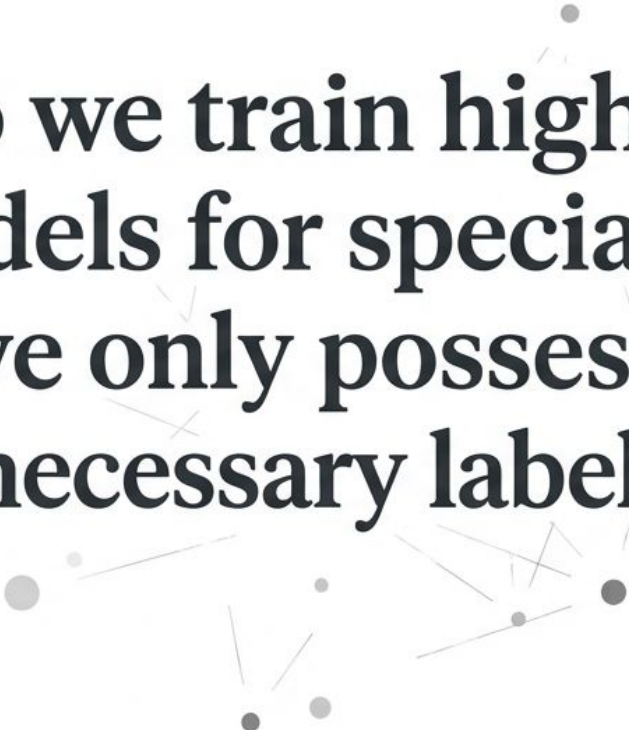


- Kaplan Scaling Laws
- Chinchilla Optimal Compute

Transfer Learning: Bridging the Data Gap

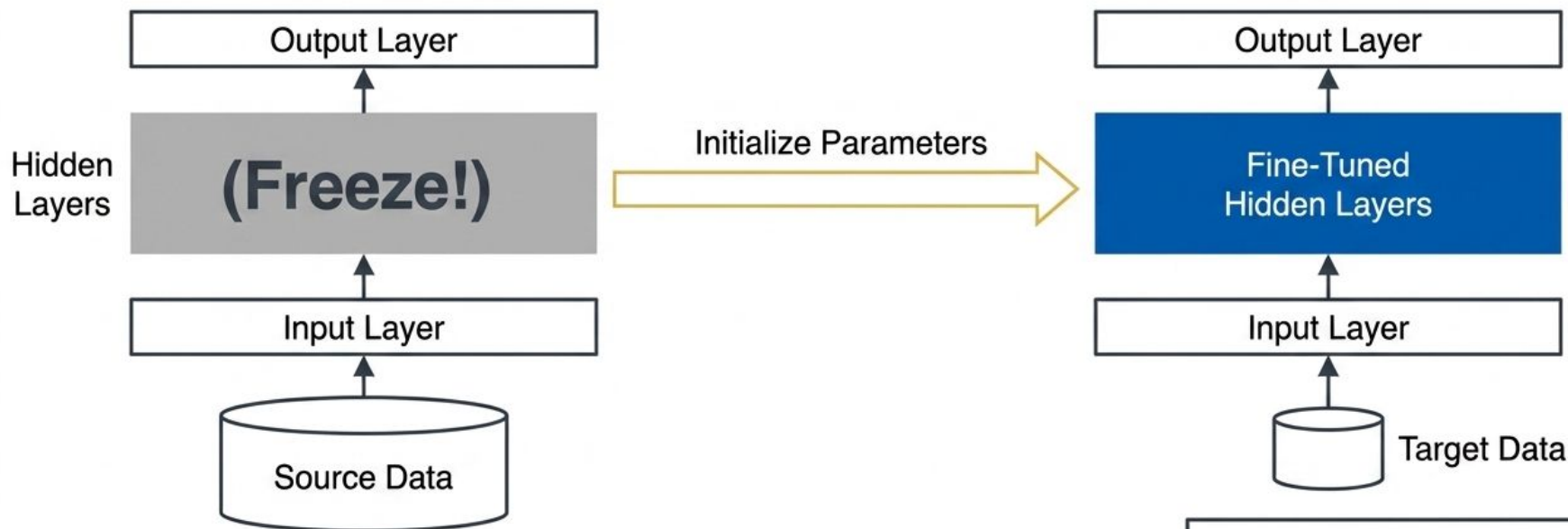


Key Takeaway: Transfer a model trained on massive source data to rapidly adapt to highly limited target data



**How do we train highly accurate
AI models for specialized tasks
when we only possess a fraction of
the necessary labeled data?**

Transfer Learning & Layer Transfer



The Mechanism

Pre-train on large data

Initialize parameters

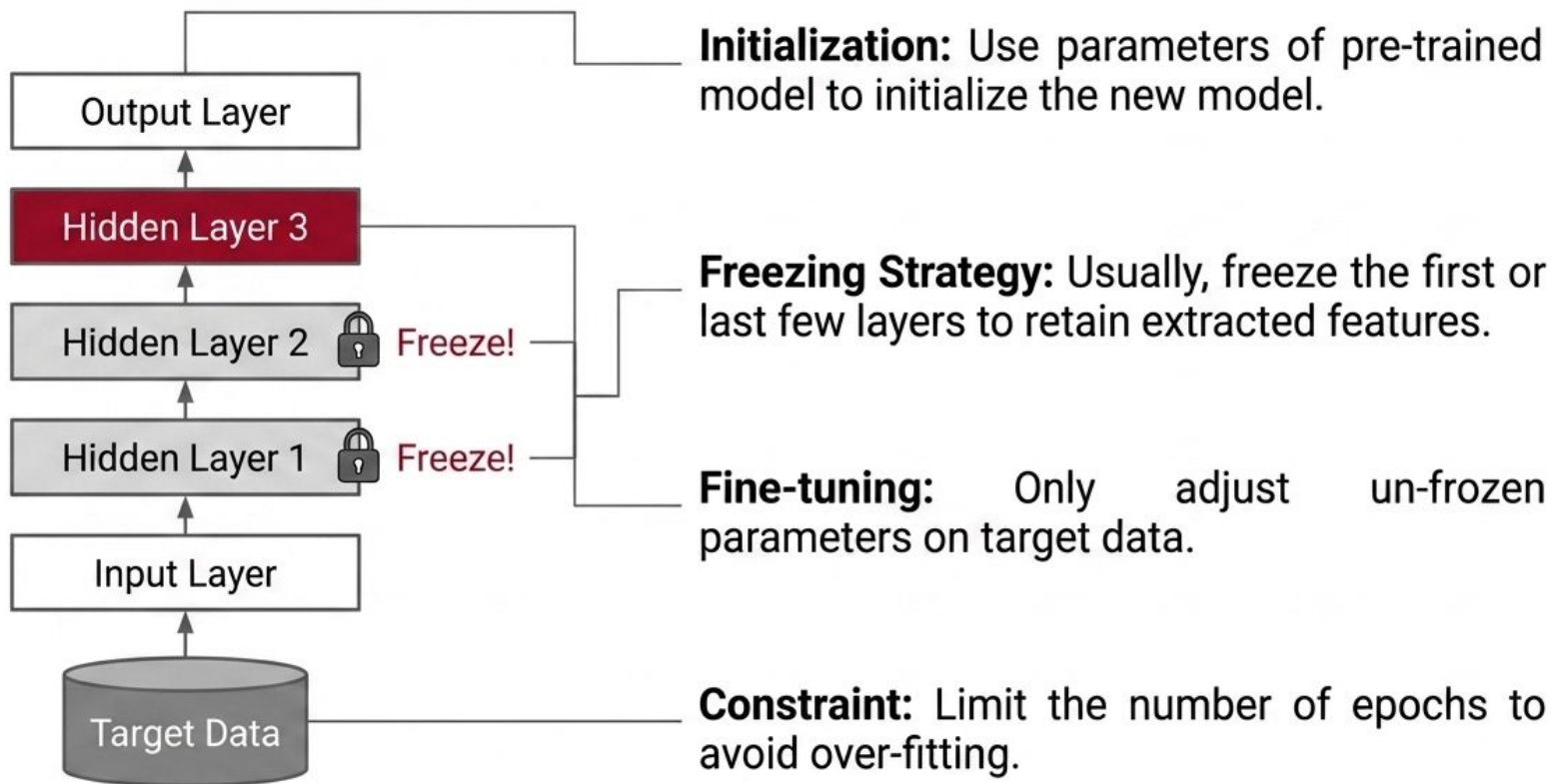
Freeze early layers

Fine-tune specific layers on target data

Open-Source Foundations

- **NLP Models:**
BERT, GPT-3
- **Computer Vision:**
VGG-16, ResNet50, ViT

Conservative Training: The Layer Transfer Approach



Foundational Open-Source Ecosystem

Utilizing open-source pre-trained models ensures highly effective and compute-efficient transfer learning across domains.

Aa

Natural Language Processing (NLP)

- **BERT**
- **GPT-3**

Optimized for sequential text extraction and generative language modeling.



Computer Vision (CV)

- **VGG-16**
- **ResNet50**
- **ViT (Vision Transformer)**

Optimized for spatial pattern recognition and complex image classification.

Multi-Task Learning (MTL) Objectives

Simultaneously undertaking multiple tasks using a single network architecture.



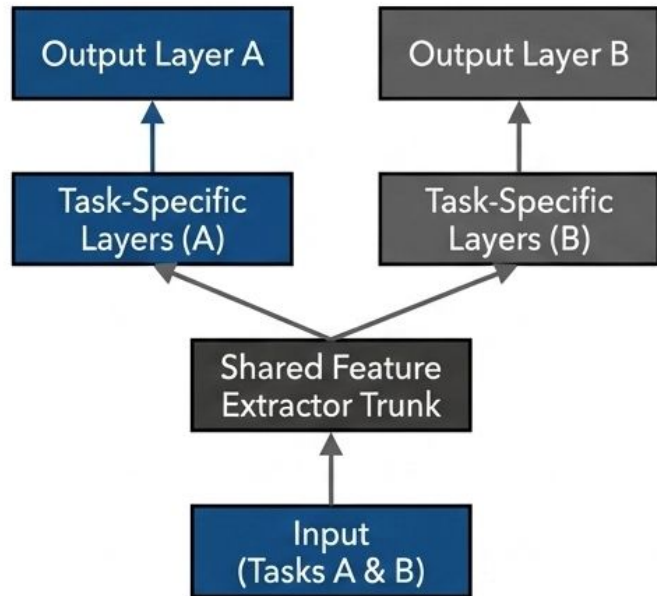
Applied Example: Cardiology

Simultaneous ECG heartbeat segmentation
AND diagnostic classification running
concurrently within one model.

Can a single neural network execute multiple distinct tasks simultaneously without exponentially multiplying our computational overhead?

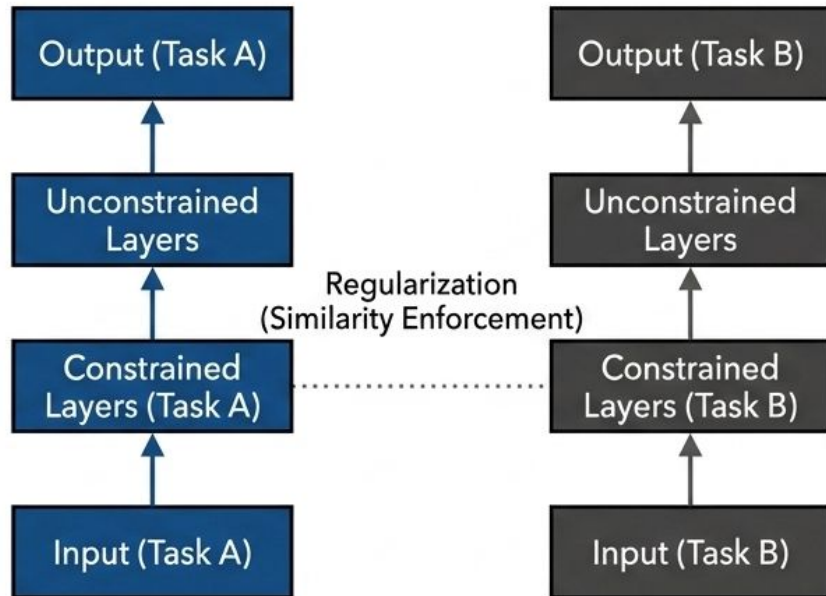
Multi-Task Learning Architectures

Hard Parameter Sharing

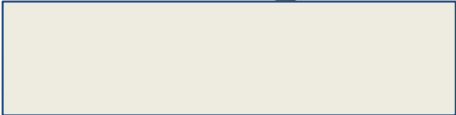


Tasks share core layers for feature extraction before splitting.

Soft Parameter Sharing

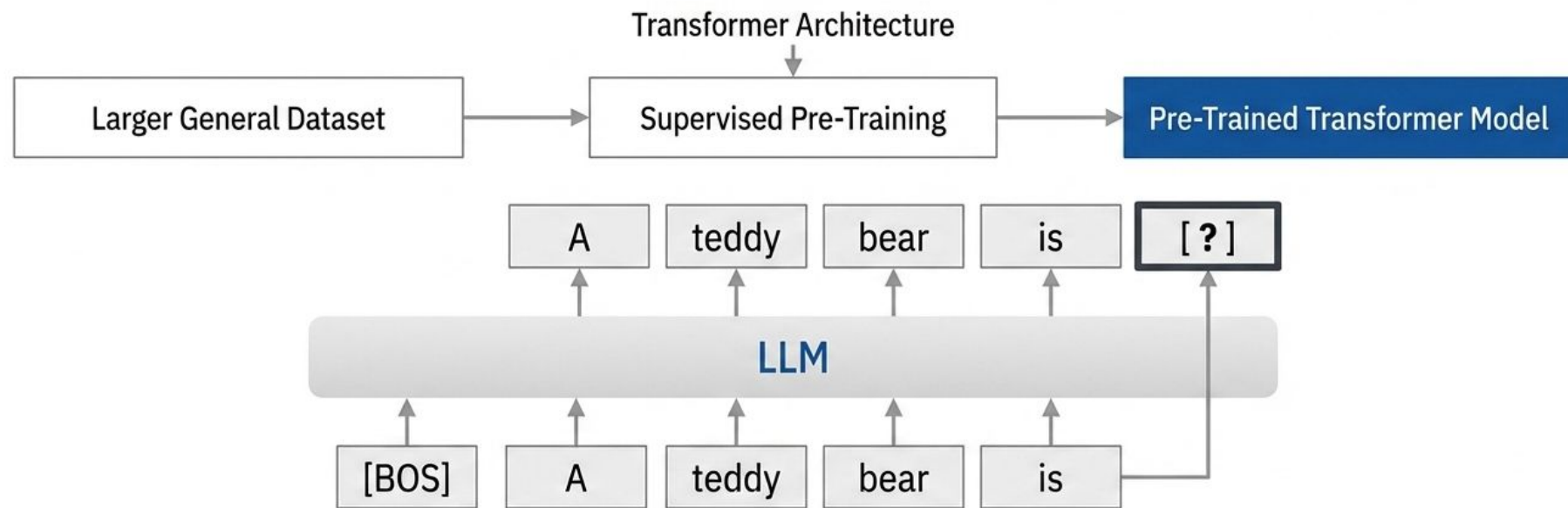


Each task maintains its own parameters, constrained by a regularization term.

Before an AI can be specialized for specific duties, what fundamental  objective is required to teach it the underlying patterns of human language?

[Token_A] → [Token_B] → [Token_C]

Transformer Pre-Training



The Scale

Training requires billions to trillions of iterative predictions.

- GPT-3: 300 Billion tokens
- LLaMA 3: 15 Trillion tokens

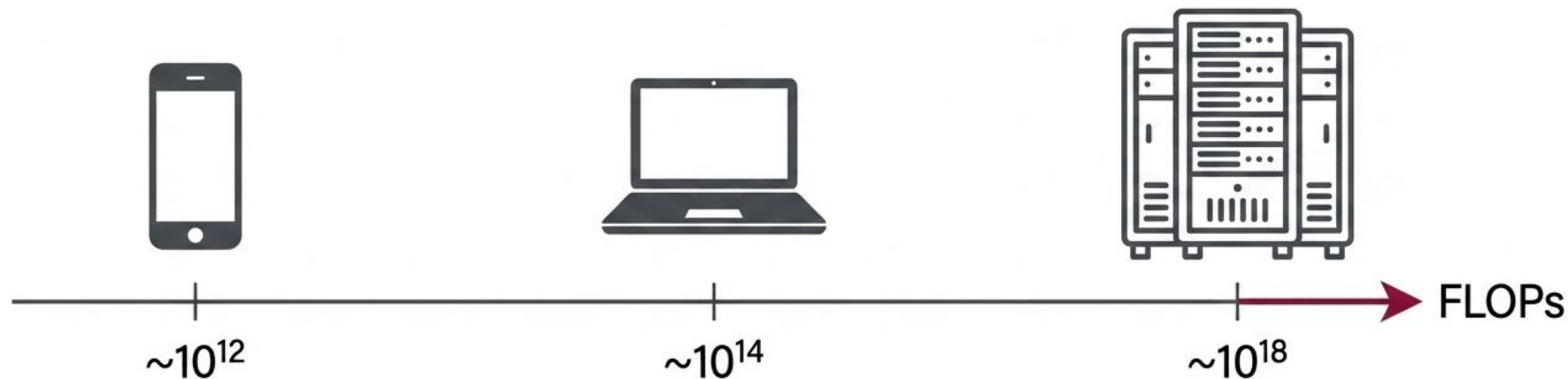
The Source

Massive open-source scraping:

- Common Crawl, Wikipedia, Reddit
- Code repositories (GitHub, StackOverflow)

Measuring Capability: The Compute Economy

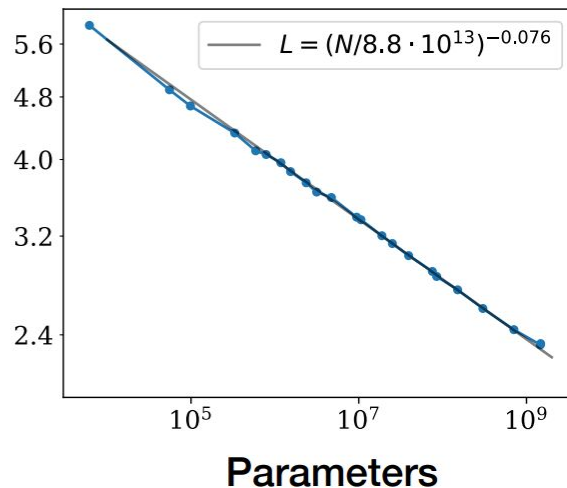
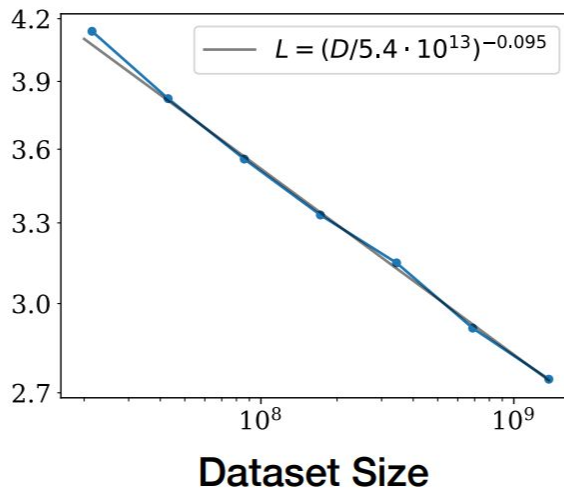
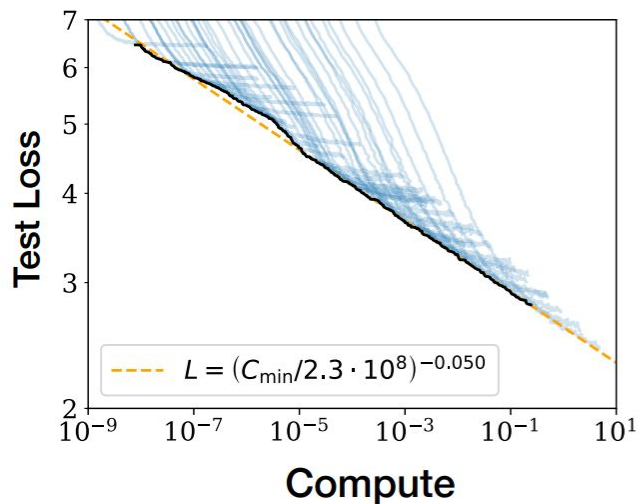
FLOPs (Floating-Point Operations Per Second): A precise unit denoting the amount of compute required and the ultimate capability of the machine.



Training Compute is a function of data volume and model size:
 $f(\text{\#tokens}, \text{\#parameters})$

If we require a vastly more capable model, is the solution simply to train it longer, or are there strict, predictable mathematical laws governing its growth?

Scaling

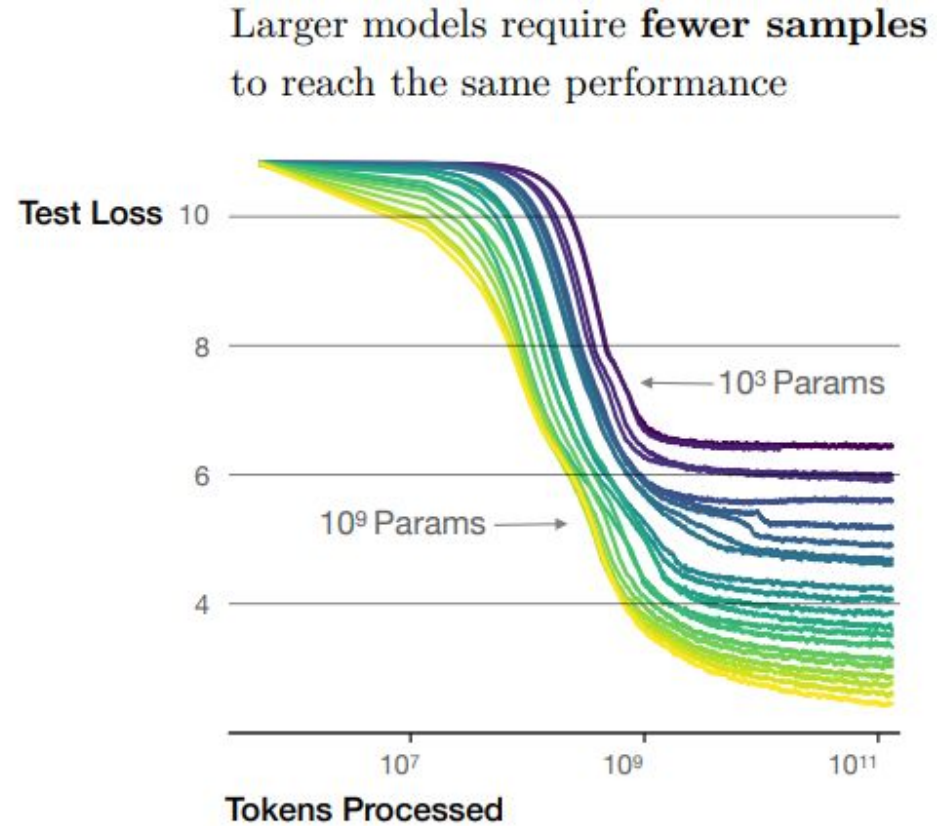


Language modelling performance improves smoothly as we increase the model size, dataset size and amount of compute.

Kaplan et. al. 2020, “Scaling Laws for Neural Language Models”

Scaling

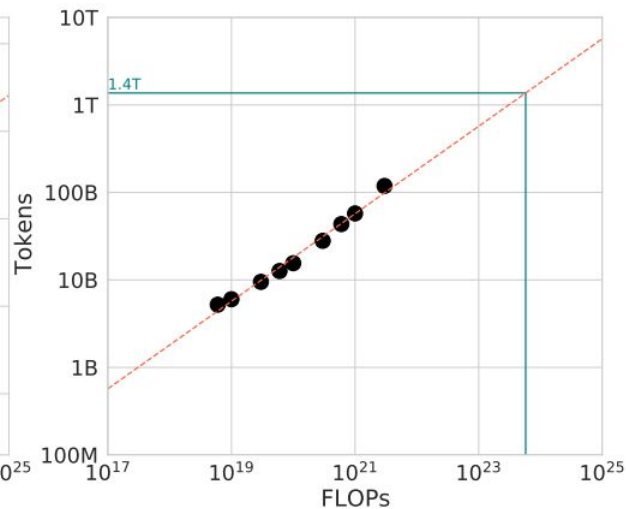
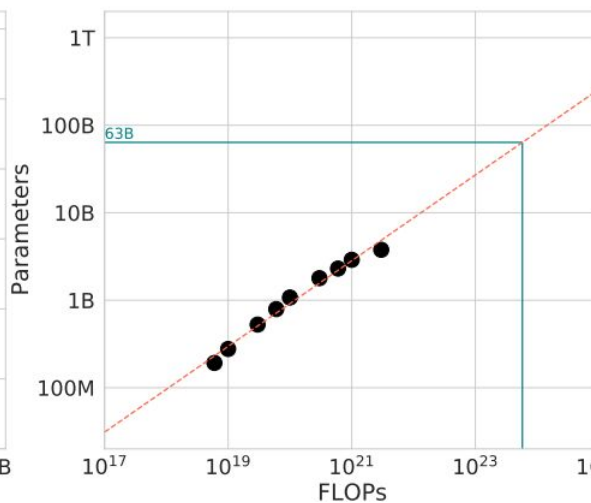
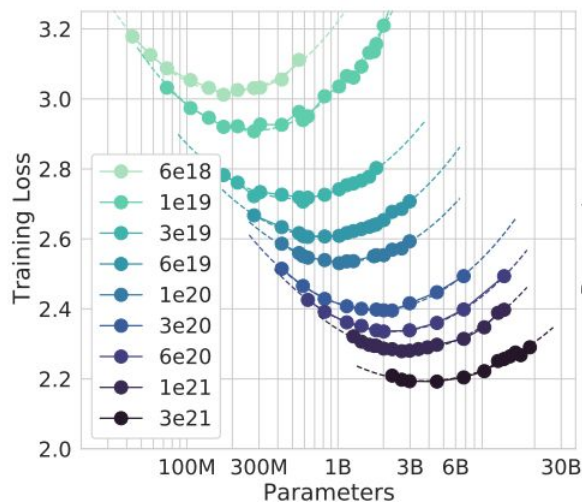
- Large models are more **sample-efficient** than small models.
- **Sample-efficient** means reaching the same level of performance with fewer optimization steps and using fewer data points.



Kaplan et. al. 2020, “Scaling Laws for Neural Language Models”

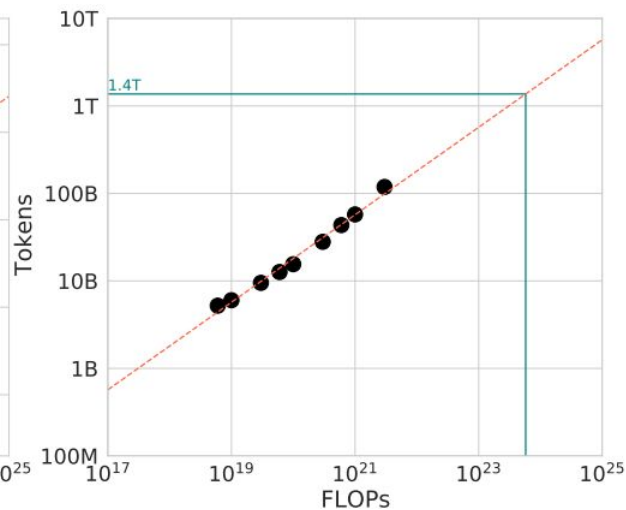
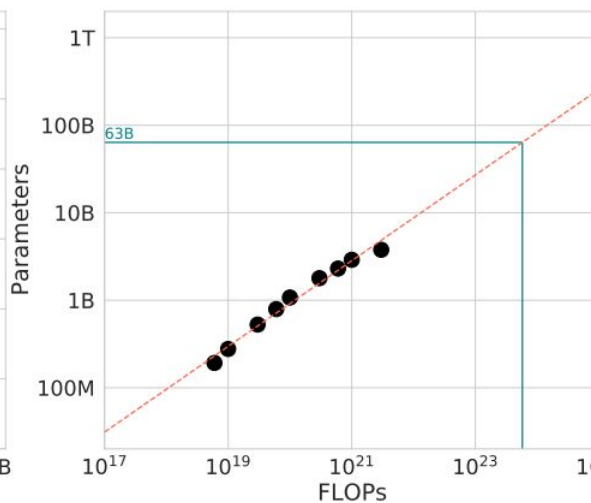
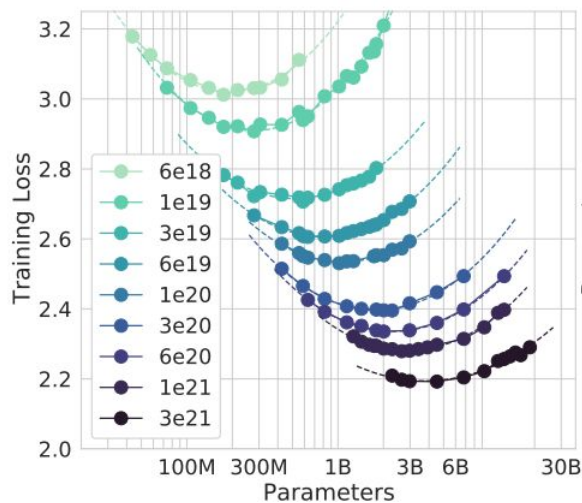
Chinchilla Law

- Each curve (left) represents a fixed compute budget.
- For a fixed compute this law tells that:
 - Very small model underfits. → High loss
 - For a too big model, tokens are not enough. → High loss
 - Sweet spot is at the bottom.



Chinchilla Law

- As compute increases $\rightarrow$ Optimal model size increases.
- As compute increases $\rightarrow$ Optimal dataset size increases.
- **Conclusion: Optimal tokens and parameters grow together.**



The Chinchilla Law: Balancing the Equation

As total compute increases, both optimal model size and optimal dataset size must increase simultaneously. Tokens and parameters must grow together.

Architecture Scenario	System Outcome
Big brain + little experience	Wasted potential. (Model is too large for the provided tokens).
Small brain + lots of experience	Cannot absorb. (Too many tokens, bottlenecked by parameter count).
Balanced Architecture	Best performance.

With computational power (FLOPs) costing millions of dollars, what is exact mathematical ratio required to prevent wasting compute on a ‘sub-optimal’ model?

The Compute Equation & The 1:20 Rule

C: Total Compute Budget
(in FLOPs).

N: Number of Parameters
(Model Size).

C

$\approx$

$6ND$

D: Number of
Training Tokens
(Dataset).

6: Operations per token per parameter
(Accounting for both forward and backward
passes).

The Golden Rule

A transformer-based LLM should be trained on approximately 20 tokens of data for every 1 parameter it possesses.

If scaling laws theoretically guarantee better performance with more data and compute, what are the physical and epistemic walls preventing infinite AI growth?

The Friction of Large Language Models

I. Economics & Ecology

Supercomputer training requires immense FLOPs ($\sim 10^{25}$ for modern LLMs). This translates directly to millions of dollars in capital expenditure and massive energy consumption.

II. Temporal Limits

The sheer volume of operations across billions of parameters necessitates an immense, unavoidable time investment to complete a single training cycle.

III. Epistemic Walls

Models suffer from strict knowledge cut-offs (e.g., Aug 31, 2025). Injecting or editing specific new knowledge post-training is exceptionally difficult.

Knowledge cutoff

Aug 31, 2025

IV. Originality Limitations

The persistent, systemic risk of plagiarism. Models can exactly reproduce portions of their seen training data, failing to generate truly novel synthesis.

The Unified Training Lifecycle

1. Pre-Training

Action: Next-token prediction on massive datasets (e.g., 15T+ tokens).

[BOS] A teddy bear is [?]

Governing Law:
Compute budgets allocated precisely via the Chinchilla Law ($C \approx 6ND$).

2. Fine-Tuning

Action: Applying Transfer Learning and Multi-Task shared parameters.

Governing Law:
Freezing foundational layers to conserve compute and adapting to limited, specialized target data.



3. Inference

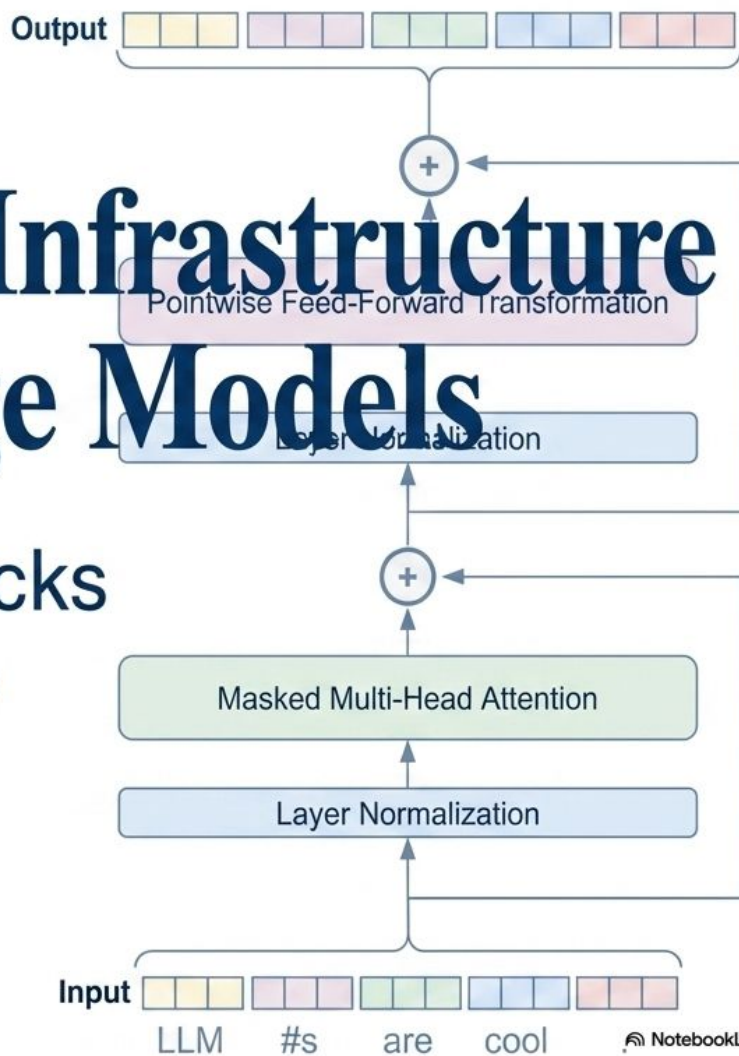
Action: Deploying the balanced, sample-efficient model into production.

Constraint:
Restricted only by the epistemic bounds of its knowledge cut-off.

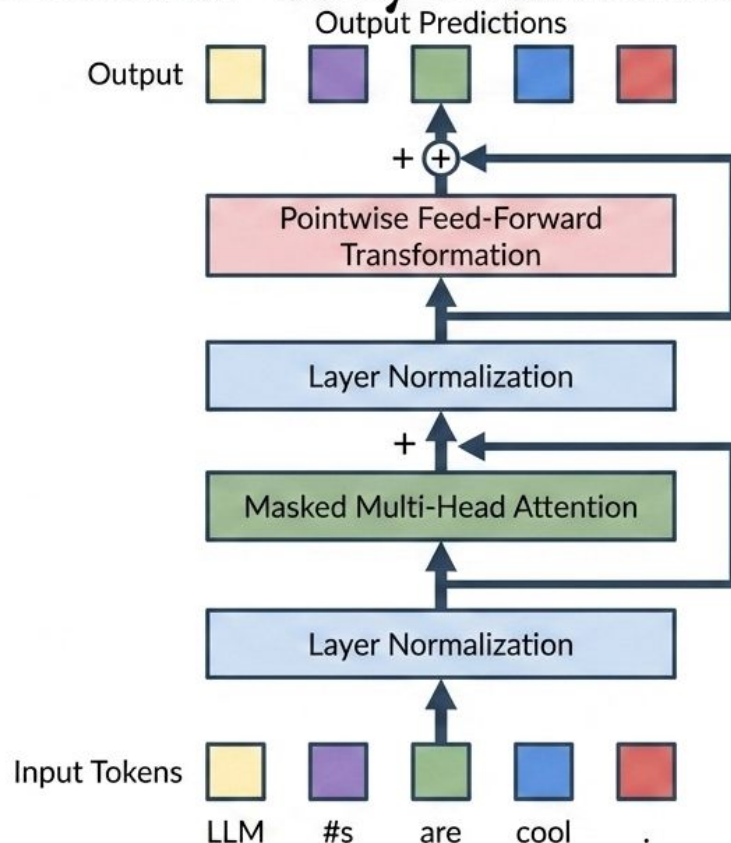
III. Training Setup and Infrastructure

Training Setup and Infrastructure for Large Language Models

Overcoming Memory Bottlenecks through **Parallelism** and **Zero Redundancy Optimization**

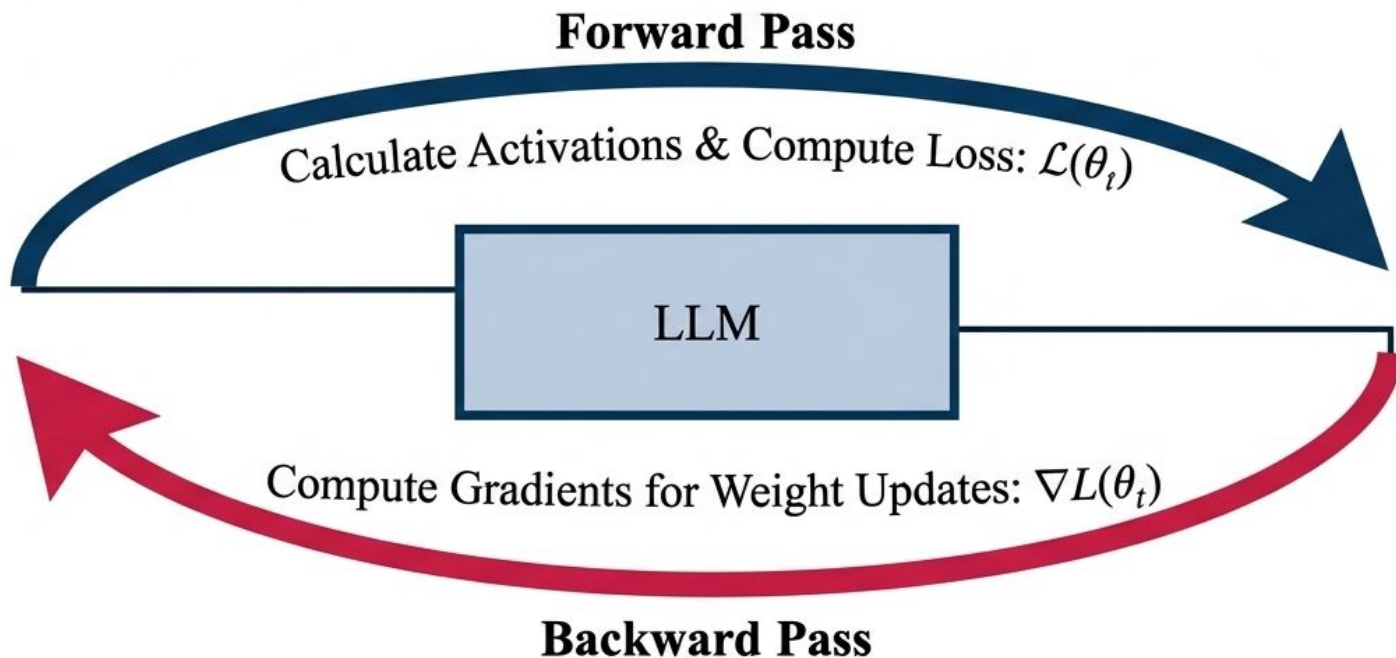


The Baseline: Decoder-Only Transformer Architecture



The Algorithmic Objective: A Two-Step Tuning Process

The goal is to tune weights to generate the optimal next token. The self-attention mechanism operates at $O(n^2)$ complexity, driving massive computational load.

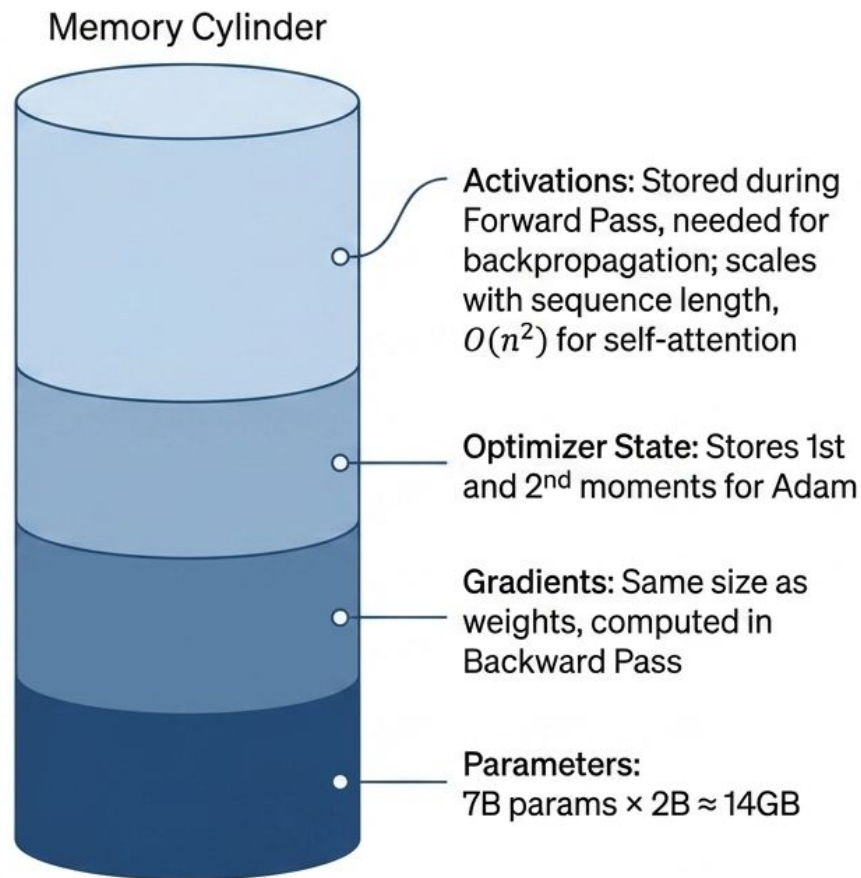


A 7-billion parameter model requires roughly 14GB of memory just to store its weights. Why does a standard GPU still run out of memory during training?



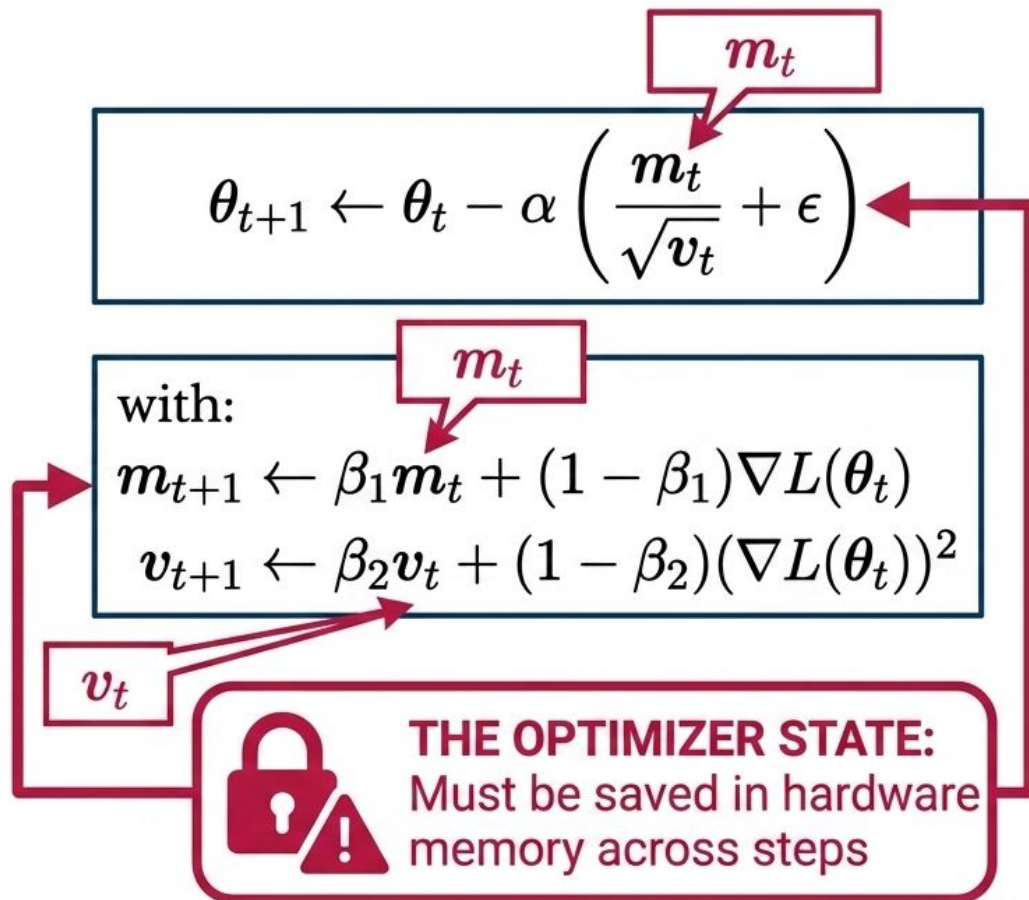
Training requires a heavy, transient memory stack.

Beyond static model parameters (weights), the Forward and Backward passes generate massive amounts of temporary data that scale with batch size and context length.



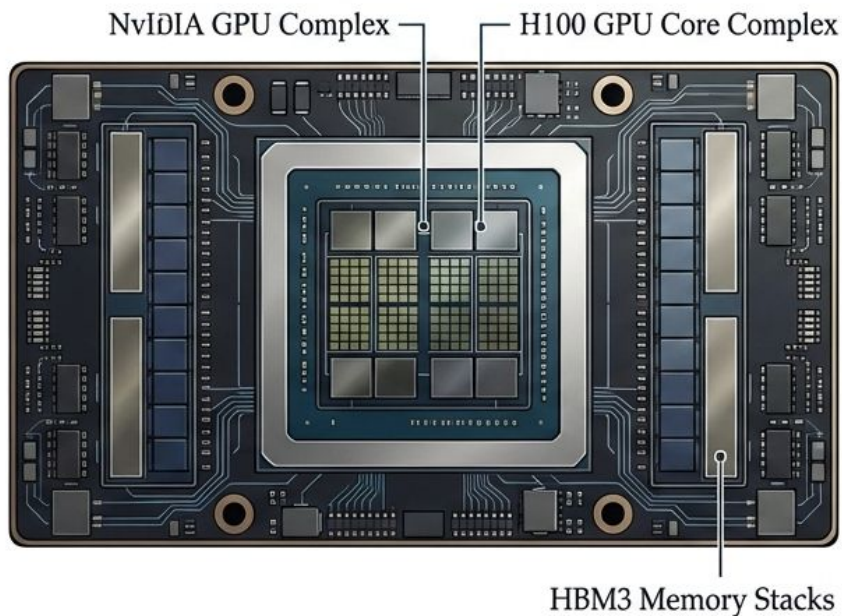
The Optimization Engine: Adam and the Cost of Memory State

- **Momentum:** Running average of past gradients.
- **Adaptive Learning Rates:** Adjusts step size per parameter.



The Physical Constraint: Compute Capacity vs. Memory Limits

Modern GPUs deliver astronomical compute speeds (TeraFLOPS), but are strictly bounded by per-GPU memory limits.



Selected Technical Specifications (H100 SXM)

TF32 Tensor Core*	989 teraFLOPS
BFLOAT16 Tensor Core*	1,979 teraFLOPS ↑
GPU Memory	80GB ⚠
GPU Memory Bandwidth	3.35TB/s

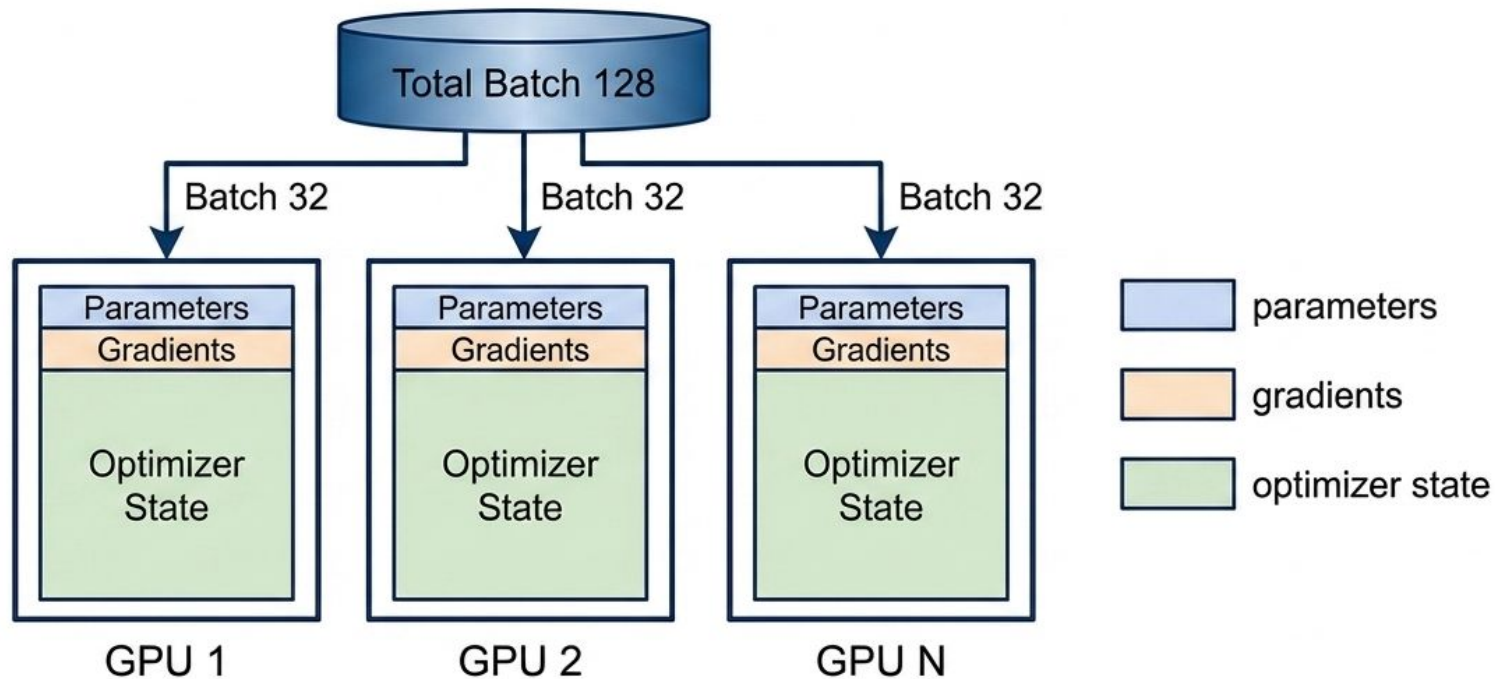
* Performance figures for specific operations as per technical datasheet.

If a single GPU cannot physically hold the entire training state, how do we distribute the computational load across a cluster?

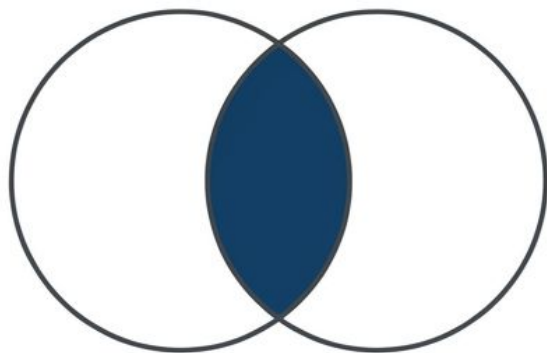


The Baseline Scaling Paradigm: Data Parallelism

Divide the data, not the model. Replicate the entire model on every GPU and split the total batch size across them.

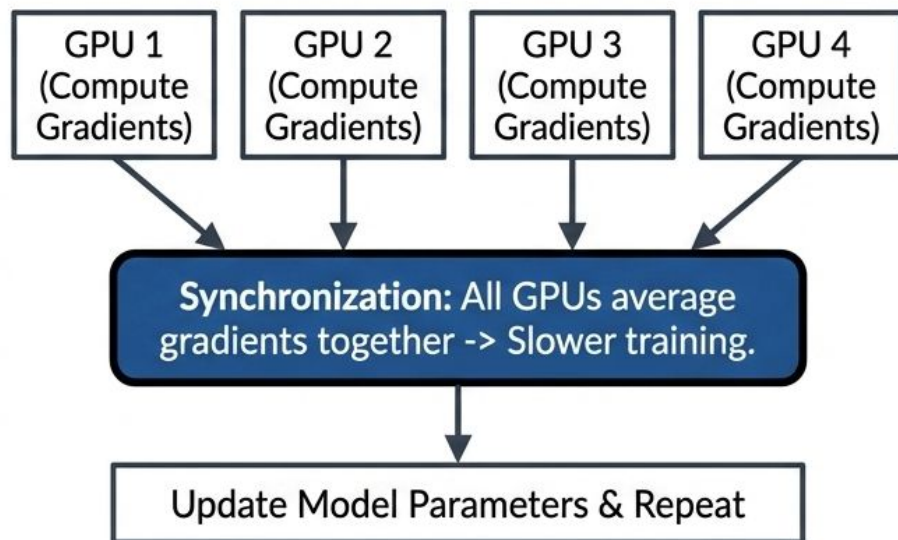
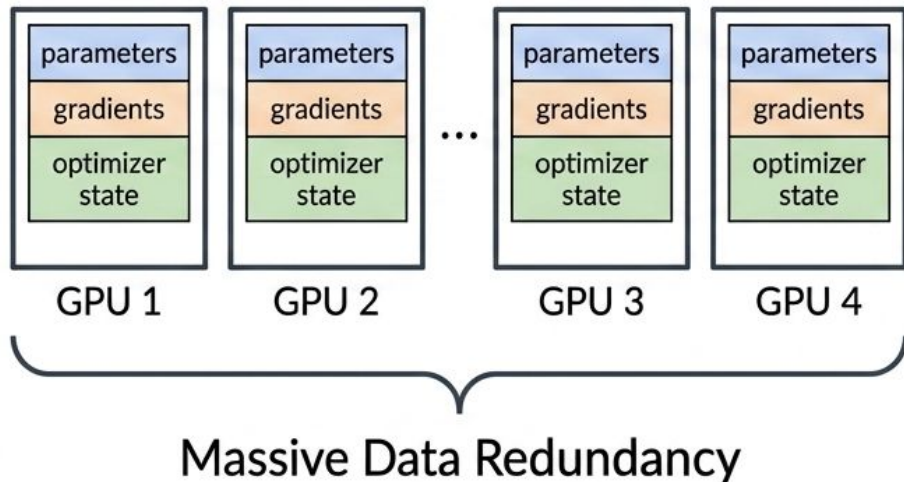


If Data Parallelism successfully divides the processing time, what is the hidden architectural cost of scaling it?



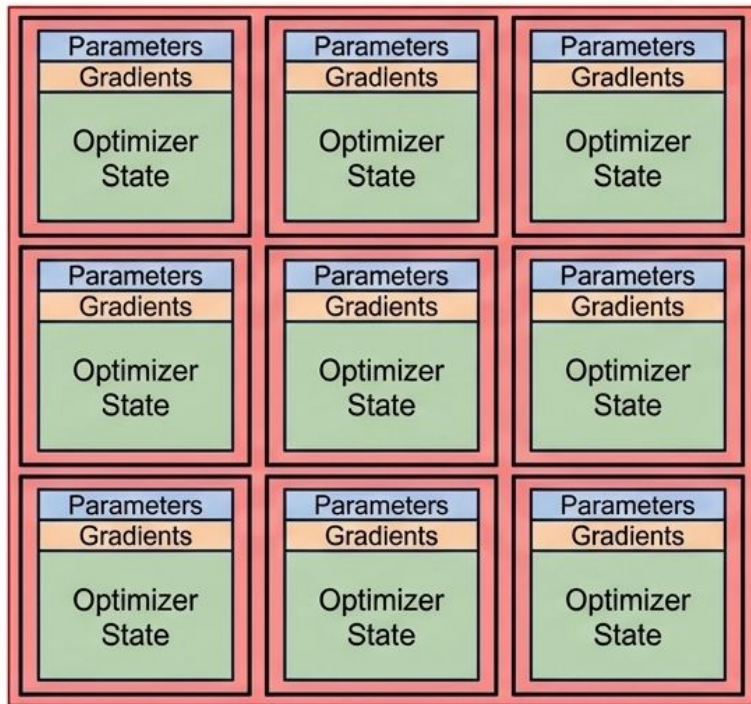
Replication creates massive redundancy and synchronization bottlenecks.

Each GPU holds an identical copy of the parameters, gradients, and optimizer state. Furthermore, a synchronization step is forced after the backward pass to **average gradients**, slowing down training.



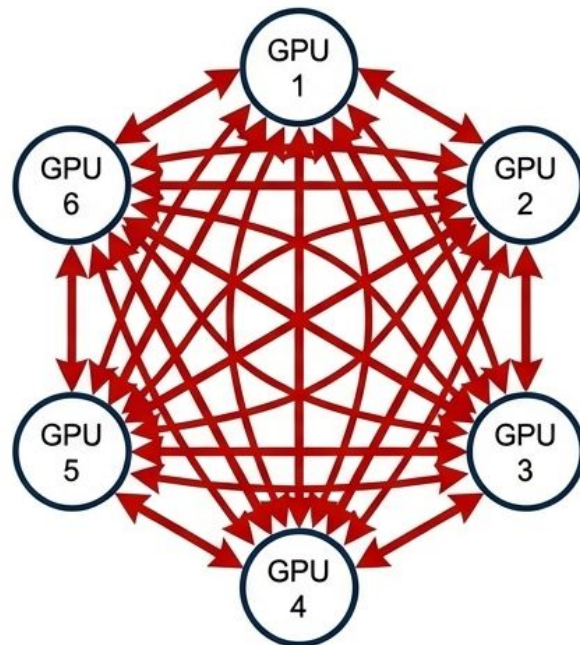
The Scaling Wall: Massive Redundancy and Synchronization Overhead

Flaw 1: Massive Redundancy



Adding 1,000 GPUs means 1,000 identical copies. Each must still fit the whole model.

Flaw 2: Synchronization Overhead



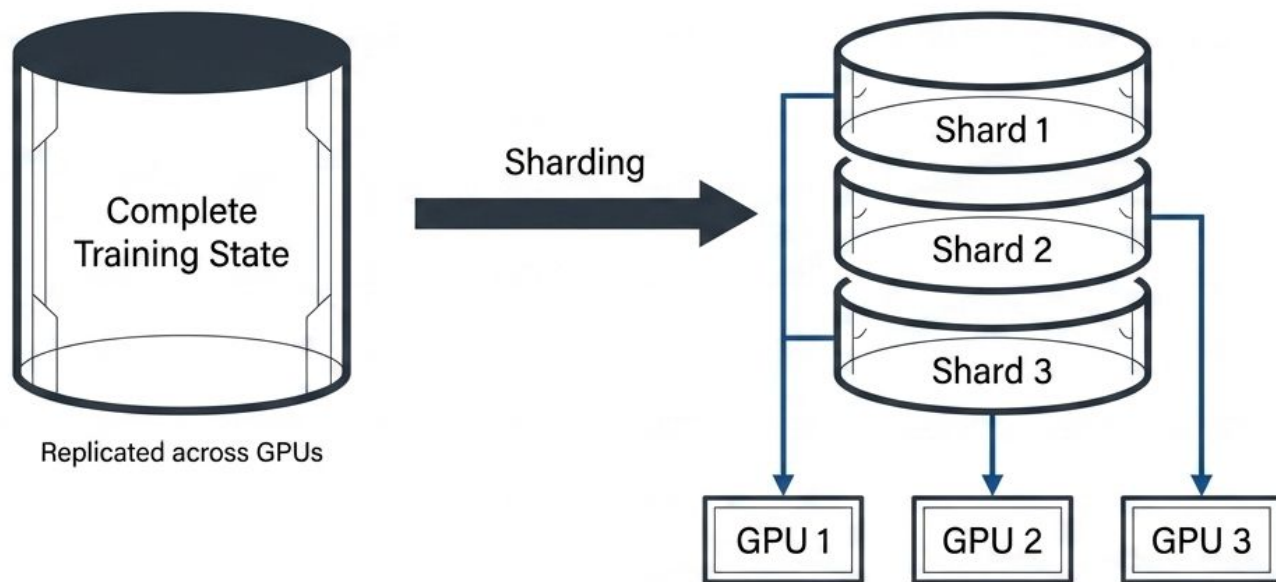
All-to-All Synchronization
GPUs must pause to average gradients locally, creating a communication bottleneck that slows training.

How can we eliminate this systemic data redundancy without sacrificing the speed of parallel processing?



ZeRO introduces database-style sharding to the training state.

Instead of replicating everything, ZeRO treats the training state like a massive database. It splits the data into manageable chunks (shards) and distributes them across the cluster.

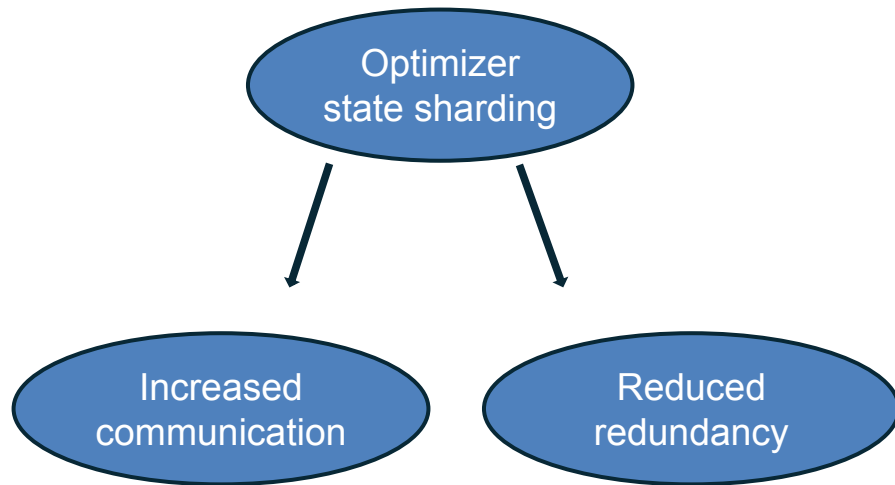


Data parallelism with ZeRO

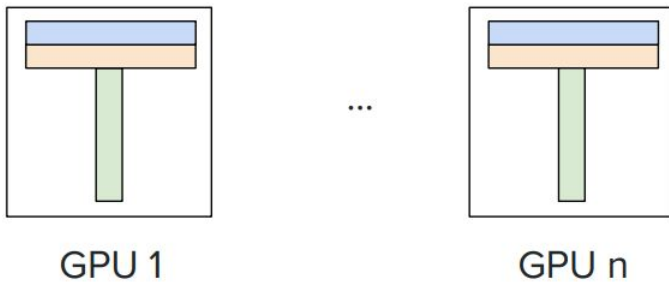
- ZeRO = Zero Redundancy Optimization
- There are three variations:
 - ZeRO-1
 - ZeRO-2
 - ZeRO-3

Data parallelism with ZeRO

- Each GPU stores all parameters.
- Each GPU stores all gradients.
- Optimizer states are sharded.

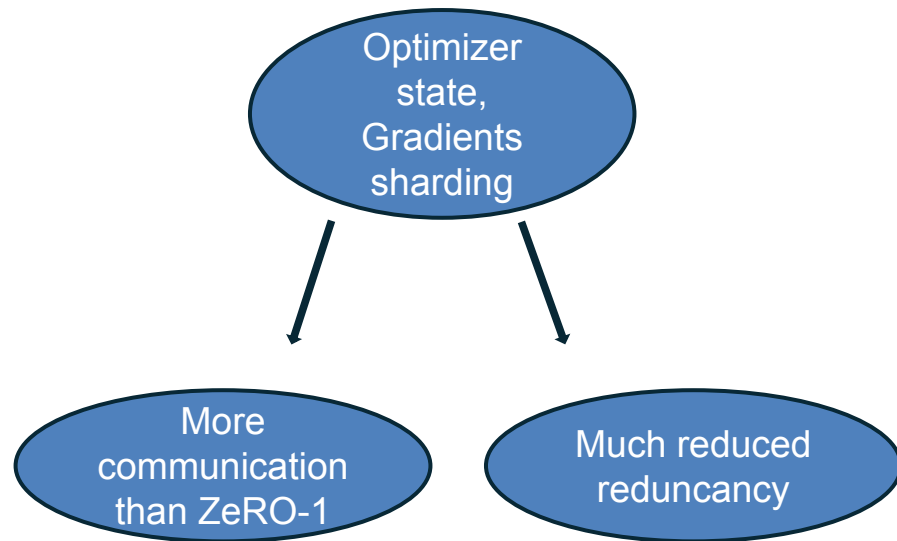


Variations. ZeRO-1: Share **optimizer state**

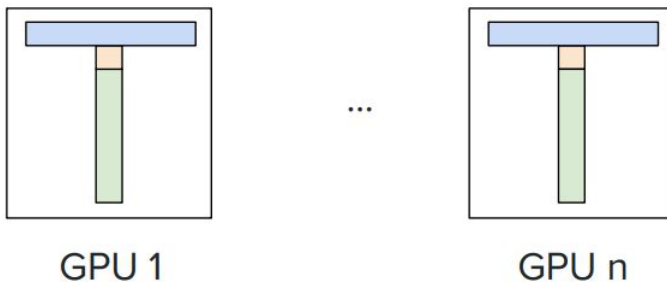


Data parallelism with ZeRO

- Each GPU stores all parameters.
- Gradients are sharded.
- Optimizer states are sharded.

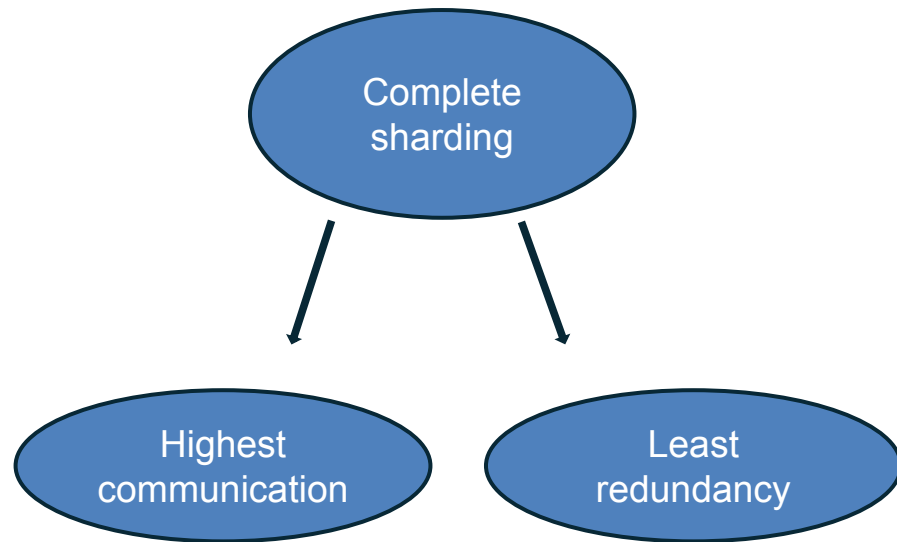


Variations. ZeRO-2: Share **optimizer state** + **gradients**

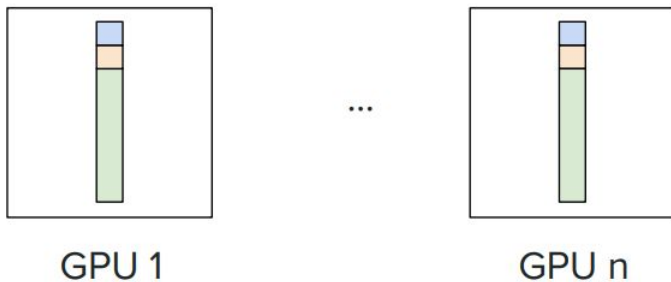


Data parallelism with ZeRO

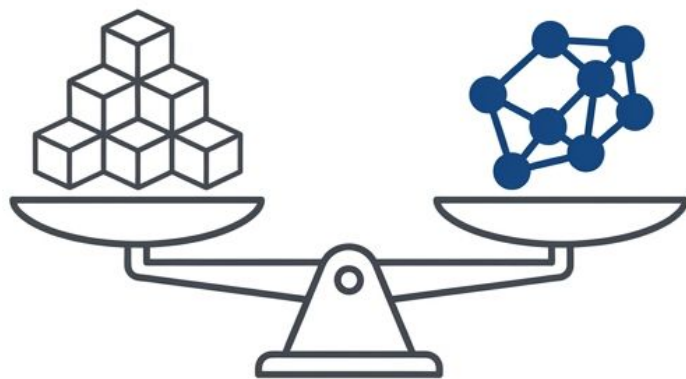
- Parameters are sharded.
- Gradients are sharded.
- Optimizer states are sharded.



Variations. ZeRO-3: Share **optimizer state** + **gradients** + **parameters**

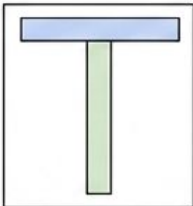
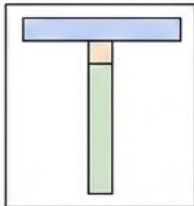
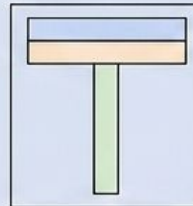


Sharding saves memory but forces GPUs to communicate to access missing pieces.
How do we balance this trade-off?



The ZeRO Optimization Matrix balances redundancy against communication overhead.

The choice of variation depends on model size and training time constraints.

	ZeRO-1	ZeRO-2	ZeRO-3
Sharded Components	 Optimizer only	 Optimizer + Gradients	 Optimizer + Gradients + Parameters
Redundancy Level	Reduced	Much Reduced	Least Redundancy
Communication Overhead	Increased	More than ZeRO-1	Highest Communication

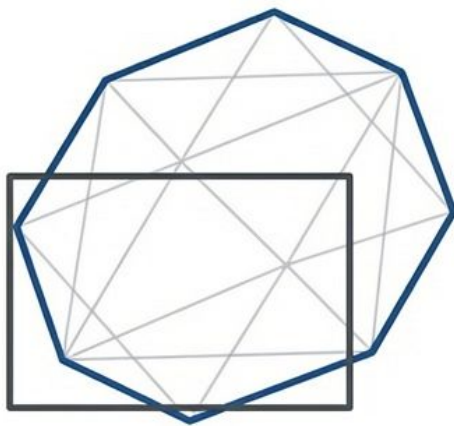
ZeRO-3 and the Ultimate Optimization Trade-Off

ZeRO-3 shards everything (Parameters, Gradients, Optimizer). The choice is a direct trade-off between model size (memory constraint) and training speed (communication overhead).

	What is Sharded?	Memory Redundancy	Communication Overhead
ZeRO-1	Optimizer State		
ZeRO-2	Optimizer + Gradients		
ZeRO-3	Optimizer + Gradients + Parameters		

Solid = High | Empty = Low/None

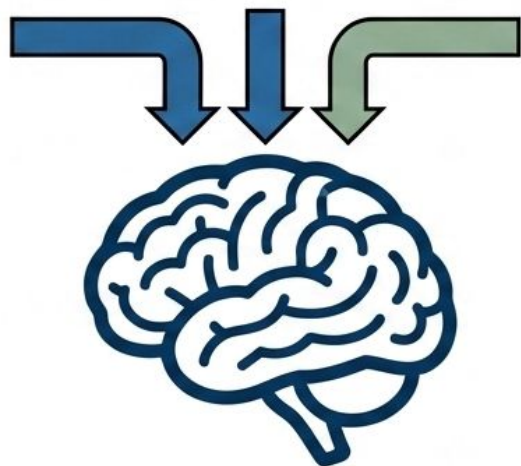
What happens when the model itself grows so large that even fully optimized data sharding cannot fit it onto a cluster?



Beyond Data: Introduction to Model Parallelism

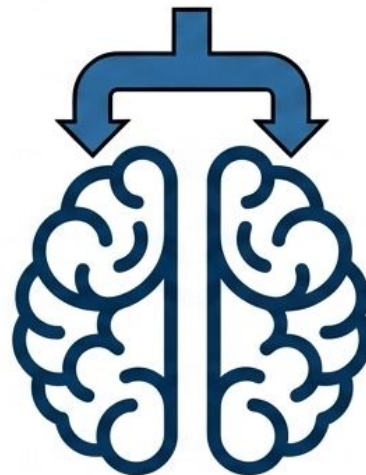
Insight: When frontier models reach billions of parameters, a single layer cannot fit on one GPU. We must split the neural network architecture itself.

Data Parallelism



Whole Model, Partial Data

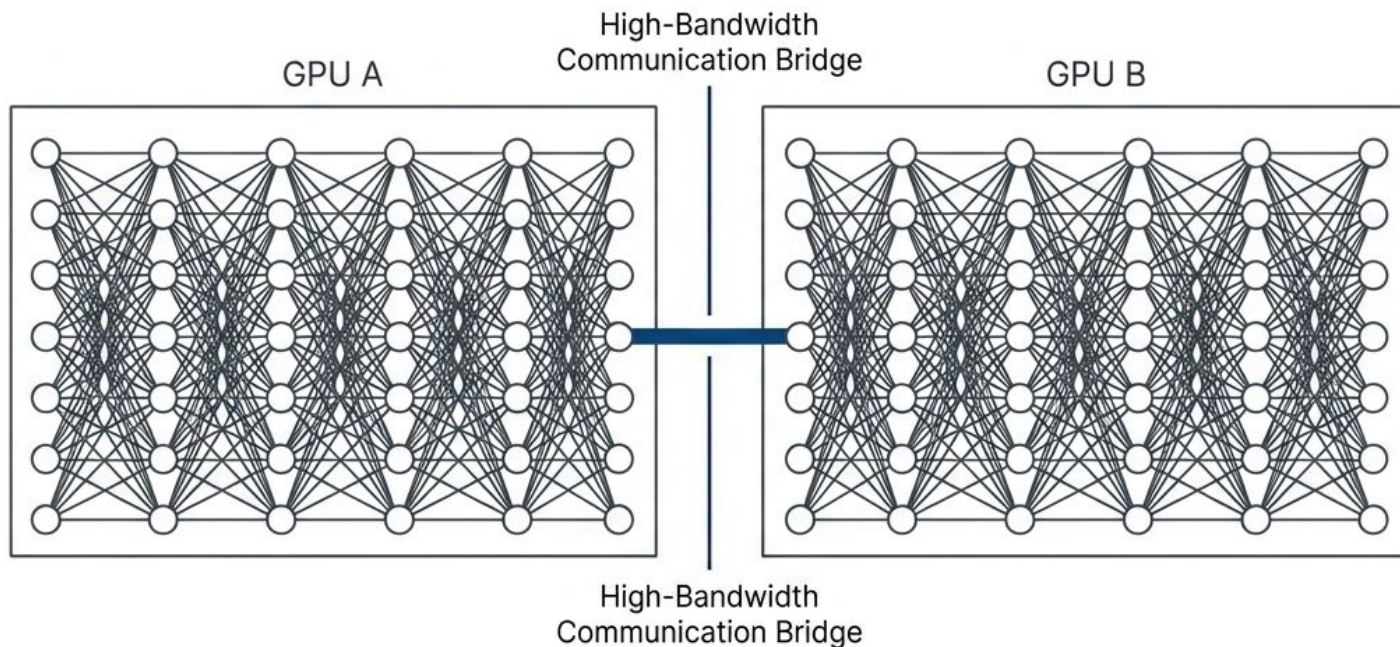
Model Parallelism



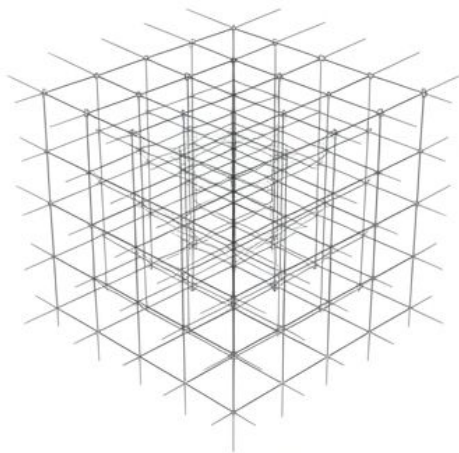
Split Model, Whole Data

Model Parallelism fractures the neural network architecture.

Instead of putting an entire multi-billion parameter model on one GPU, the math and architecture of the model itself are sliced and distributed across multiple GPUs.



A neural network is a complex sequence of layers, attention mechanisms, and routing.
Where exactly do we make the cuts?



The Taxonomy of Architectural Fracturing

We can slice the model across different dimensional planes based on where the bottleneck occurs.



Tensor Parallelism

Slices individual layers/matrix multiplication across GPUs. Requires frequent communication.



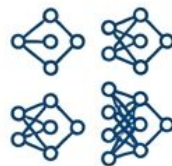
Pipeline Parallelism

Splits across the sequence length dimension (e.g., layers 1-10 on GPU 1, layers 11-20 on GPU 2).



Context Parallelism

Distributes different parts of a long context window across GPUs, collaborating during attention computation.

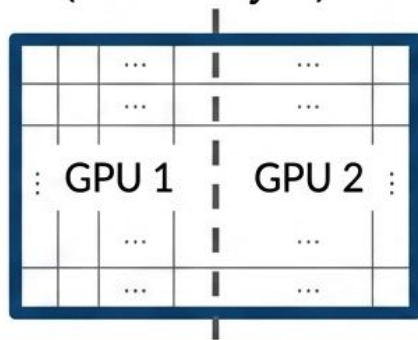


Expert Parallelism

For MoE models; hosts different sub-networks (experts) on different GPUs.

Architectural Splitting: Intra-Layer vs. Inter-Layer Parallelism

Tensor Parallelism (Intra-Layer)

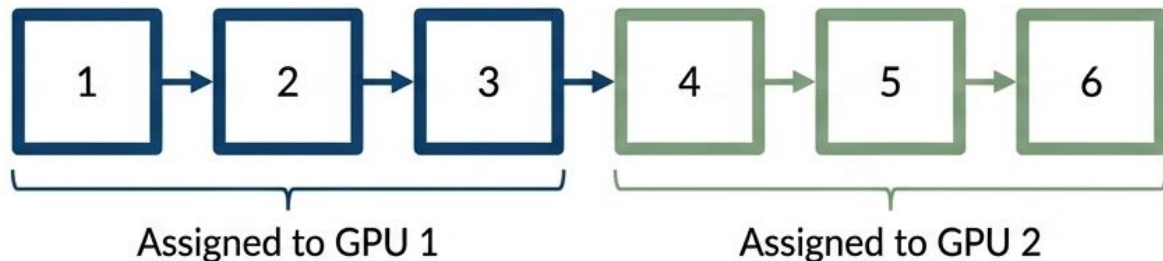


Splits the matrix multiplication math inside individual layers.

Requires intense, frequent communication.



Pipeline Parallelism (Inter-Layer)



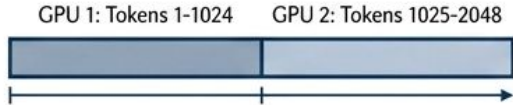
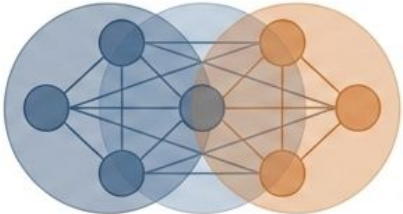
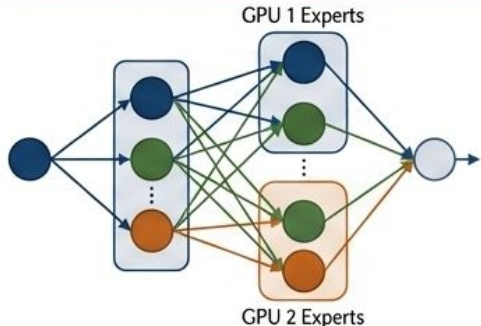
Splits the model by layers sequentially.

Risks latency via GPU idle time.



Advanced Paradigms: Sequence, Context, and Expert Parallelism

The Academic Blueprint

Sequence Parallelism	Context Parallelism	Expert Parallelism
		
Splits across sequence length dimension. Example: Tokens 1-1024 on GPU 1; 1025-2048 on GPU 2.	Crucial for long-context models. GPUs hold different parts of the context and collaborate intensely during attention computation.	Used in Mixture of Experts (MoE) models. Different GPUs physically host distinct sub-network “experts”.